

What is JavaScript & how it runs

What is JavaScript & how it runs

ما هي JavaScript وكيف تعمل؟

أولاً: ما هي JavaScript؟

JavaScript هي لغة برمجة:

- بتشغل داخل المتصفح
- بتخلي صفحات الويب **تفاعلية**
- بتتعامل مع:
 - الضغط على الأزرار
 - إدخال المستخدم
 - تغيير محتوى الصفحة
 - جلب بيانات من API

مثال بسيط:

بدون JavaScript:

صفحة ثابتة بس بتعرض

مع JavaScript:

صفحة بترد عليك وتتحرك وتتغير حسب تصرفك

دور JavaScript في الويب

اللغة	الدور
HTML	هيكل الصفحة
CSS	الشكل والتصميم
JavaScript	السلوك والتفاعل

تشبيه مشهور:

- HTML = الهيكل العظمي
- CSS = الجلد والملابس
- JavaScript = العقل والحركة

أين تعمل JavaScript؟

1 داخل المتصفح (Client-side)

- Chrome
- Firefox

• Edge

بتشغل باستخدام:

• **JavaScript Engine**

○ Chrome → V8

○ Firefox → SpiderMonkey

2. خارج المتصفح (Server-side)

• باستخدام **Node.js**

• نفس اللغة بس بيئة مختلفة

كيف تعمل JavaScript داخل المتصفح؟ (ببساطة)

خاينا نفترض إن عندك كود JS:

```
console.log("Hello World");
```

اللي بيحصل:

1. المتصفح يشغل **JavaScript Engine**

2. الـ Engine:

○ يقرأ الكود

○ يفسّره

○ ينقّذه سطر سطر

3. النتيجة تظهر في الـ Console

JavaScript Language Type

● Interpreted Language

○ مش محتاجة Compile قبل التشغيل

● Single-threaded

○ بتنّفذ مهمة واحدة في نفس الوقت

● Non-blocking

○ بتستخدم Event Loop عشان ما تعطلش الصفحة

JavaScript Execution Flow (تصور مبسط)

1. تحميل ملف HTML

2. تحميل CSS

3. تحميل JavaScript

4. تنفيذ JavaScript من فوق لتحت

5. انتظار أي (Events (click – submit – load

مثال عملي بسيط

HTML

```
<button onclick="sayHi()">Click me</button>
```

JavaScript

```
function sayHi() {  
    alert("Hello from JavaScript!");  
}
```

هنا JavaScript:

- سمعت (Click Event)
- نفذت Function
- تفاعلت مع المستخدم

JavaScript & The Browser Environment

JavaScript تقدر تتعامل مع:

- DOM (عناصر الصفحة)
- BOM (window, location, history)
- Events
- Storage (LocalStorage)

نقاط مهمة لازم الطالب يفهمها

- JavaScript لغة أساسية لأي Frontend Developer
- هي اللي بتخلي الموقع "حي"
- نفس اللغة بتشتغل Frontend & Backend

اختر الإجابة الصحيحة (MCQ)

1 JavaScript بتستخدم في إيه؟

- (A) تصميم الصفحة
- (B) إنشاء هيكل الصفحة
- (C) إضافة التفاعل والسلوك
- (D) إنشاء الصور

2 JavaScript بتشتغل فيين؟

- (A) السيرفر فقط
- (B) المتصفح فقط
- (C) المتصفح والسيرفر
- (D) قاعدة البيانات

3 أي جزء مسؤول عن تشغيل JavaScript في المتصفح؟

- A) HTML Parser
- B) CSS Engine
- C) JavaScript Engine
- D) DOM Tree

First: What is JavaScript?

JavaScript is a programming language:

- Works inside the browser
- Makes web pages interactive
- Handles:
 - Button clicks
 - User input
 - Changing page content
 - Fetching data from APIs

Simple example:

Without JavaScript:

- Static page just displayed

With JavaScript:

- Page responds, moves, and changes according to your actions

JavaScript's Role in the Web

Language	Role
HTML	Page structure
CSS	Appearance & design
JavaScript	Behavior & interaction

Famous analogy:

- HTML = Skeleton
- CSS = Skin & Clothes
- JavaScript = Brain & Movement

Where does JavaScript run?

1 Inside the browser (Client-side)

- Chrome
- Firefox
- Edge
- Runs using:
JavaScript Engine

- Chrome → V8
- Firefox → SpiderMonkey

2 Outside the browser (Server-side)

- Using Node.js
- Same language, different environment

How JavaScript works inside the browser? (Simply)

Suppose you have JS code:

```
console.log("Hello World");
```

What happens:

- Browser runs JavaScript Engine
- The Engine:
 - Reads the code
 - Interprets it
 - Executes line by line
- Result shows in the Console

JavaScript Language Type

- Interpreted Language → No need to Compile before running
- Single-threaded → Executes one task at a time

- Non-blocking → Uses Event Loop so it doesn't block the page

JavaScript Execution Flow (Simplified)

- Load HTML file
- Load CSS
- Load JavaScript
- Execute JavaScript top to bottom
- Wait for any Events (click – submit – load)

Simple Practical Example

HTML:

```
<button onclick="sayHi()">Click me</button>
```

JavaScript:

```
function sayHi() {  
    alert("Hello from JavaScript!");  
}
```

}

Here JavaScript:

- Listened to Event (Click)
- Executed Function
- Interacted with the user

JavaScript & The Browser Environment

JavaScript can handle:

- DOM (Page elements)
- BOM (window, location, history)
- Events
- Storage (LocalStorage)

Important points students must understand

- JavaScript is essential for any Frontend Developer
- It makes the website “alive”
- Same language works Frontend & Backend

Choose the correct answer (MCQ)

① JavaScript is used for?

- A) Designing the page
- B) Creating page structure
- C) Adding interaction & behavior
- D) Creating images

② Where does JavaScript run?

- A) Server only
- B) Browser only
- C) Browser & Server
- D) Database

③ Which part is responsible for running JavaScript in the browser?

- A) HTML Parser

B) CSS Engine

C) JavaScript Engine

D) DOM Tree

Variables

Variables in JavaScript

JavaScript المتغيرات (var – let – const) في

يعني إيه Variable؟

المتغير هو مكان في الذاكرة بنخزن فيه قيمة ونستخدمها بعدين.

تشبيه بسيط:

المتغير = صندوق

القيمة = الحاجة اللي جوّه الصندوق

ليه بنستخدم المتغيرات؟

- تخزين بيانات
- إعادة استخدام القيم
- كتابة كود أوضح وأسهل

طرق تعريف المتغيرات في JavaScript

عندنا 3 طرق:

1. `var`

2. `let`

3. `const`

1 `var` (الطريقة القديمة)

```
var name = "Youssef";  
var age = 24;
```

مشاكل `var`:

- **Scope** مش واضح
- ممكن يتعاد تعريفه
- بيعمل مشاكل في المشاريع الكبيرة

مثال:

```
var x = 10;  
var x = 20; // مفيش مشكلة ❌
```

عشان كده استخدمه **غير مفضل** حاليًا.

2 let (الطريقة الحديثة)

```
let city = "Cairo";  
city = "Alex";
```

مميزات let:

- Block Scope
- ينفع تغيير قيمته
- ما ينفعش تعرّفه مرتين في نفس الـ scope

مثال:

```
let y = 10;  
let y = 20; // ❌ Error
```

3 const (القيمة الثابتة)

```
const PI = 3.14;
```

مميزات const:

- لا يمكن تغيير القيمة
- لازم يكتب معاه قيمة
- Block Scope

مثال خطأ:

```
const x;  
x = 5; // ❌ Error
```

ملاحظة مهمة جدًا عن `const`

`const` مش معناها إن القيمة ما تتغيرش دائمًا
لكن معناها إن المرجع نفسه ثابت

مثال Object:

```
const user = {  
  name: "Ali"  
};
```

```
user.name = "Ahmed"; // ✅ مسموح
```

لكن:

```
user = {}; // ❌ ممنوع
```

Scope يعني إيه؟

:Block Scope

```
if (true) {  
  let a = 5;  
  const b = 10;
```

```
}  
console.log(a); // ❌ Error
```

:Function Scope

```
function test() {  
  var x = 10;  
}  
console.log(x); // ❌ Error
```

لكن:

```
if (true) {  
  var z = 10;  
}  
console.log(z); // ✅
```

وده سبب خطورة **var**.

مقارنة سريعة

const	let	var	خاصية
Block	Block	Function	Scope
❌	❌	✅	ينفع يتعاد تعريفه
❌	✅	✅	ينفع تتغير قيمته

✓	✓	✗	استخدامه حاليًا
---	---	---	-----------------

Best Practice (قاعدة مهمة)

استخدم:

- `const` دائمًا
- لو احتجت تغيير القيمة → `let`
- أبعد عن `var`

أخطاء شائعة

- استخدام `var` في كود حديث
- نسيان إعطاء قيمة لـ `const`
- محاولة تغيير قيمة `const`
- تعريف متغيرات بنفس الاسم

اختر الإجابة الصحيحة (MCQ)

1 أي متغير لا يمكن تغيير قيمته؟

- A) var
- B) let
- C) const
- D) all

2 أي نوع Scope خاص بـ let و const؟

- A) Global
- B) Function
- C) Block
- D) File

3 هل الكود فيه مشكلة؟

```
const x = 10;  
x = 20;
```

- A) صحيح
- B) خطأ

What is a Variable?

A variable is a place in memory where we store a value and use it later.

Simple analogy:

- Variable = Box
- Value = The thing inside the box

Why do we use variables?

- Store data
- Reuse values
- Write clearer and easier code

Ways to define variables in JavaScript

We have 3 ways:

- `var`

- `let`
- `const`

1 `var` (old way)

```
var name = "Youssef";  
var age = 24;
```

Problems with `var`:

- Scope is not clear
- Can be re-declared
- Causes issues in big projects

Example:

```
var x = 10;  
var x = 20; // No problem ❌
```

So, its use is not recommended nowadays.

2 let (modern way)

```
let city = "Cairo";  
city = "Alex";
```

Advantages of let:

- Block Scope
- Can change its value
- Cannot be declared twice in the same scope

Example:

```
let y = 10;  
let y = 20; // ❌ Error
```

3 const (constant value)

```
const PI = 3.14;
```

Advantages of const:

- Cannot change the value
- Must be assigned a value
- Block Scope

Wrong example:

```
const x;
```

```
x = 5; // ❌ Error
```

Very important note about const

`const` does not always mean the value can't change, it means the reference is fixed.

Example with Object:

```
const user = {  
  name: "Ali"  
};
```

```
user.name = "Ahmed"; //  Allowed
```

```
user = {}; //  Not allowed
```

What is Scope?

Block Scope:

```
if (true) {  
  let a = 5;  
  const b = 10;  
}  
  
console.log(a); //  Error
```

Function Scope:

```
function test() {
```

```
    var x = 10;
  }
console.log(x); // ❌ Error
```

But:

```
if (true) {
  var z = 10;
}
console.log(z); // ✅
```

This is why **var** can be dangerous.

Quick Comparison

Feature	var	let	const
Scope	Function	Block	Block

Can be re-declared			
Can change value			
Use nowadays			

Best Practice

- Always use `const`
- If you need to change the value → use `let`
- Avoid `var`

Common Mistakes

- Using `var` in modern code

- Forgetting to assign a value to `const`
- Trying to change a `const` value
- Declaring variables with the same name

Choose the correct answer (MCQ)

1 Which variable cannot change its value?

- A) var
- B) let
- C) const
- D) all

2 Which scope is for let & const?

- A) Global
- B) Function
- C) Block
- D) File

3 Is the code correct?


```
const x = 10;
```

```
x = 20;
```

A) Correct

B) Wrong

Data Types

Data Types in JavaScript

JavaScript أنواع البيانات في

يعني إيه Data Type؟

نوع البيانات هو اللي بيحدد:

- القيمة دي إيه؟
- نقدر نعمل عليها إيه؟

مثال:

```
10      // رقم  
"Hello" // نص  
true    // قيمة منطقية
```

JavaScript لغة Dynamic

يعني:

- مش محتاج تحدد نوع البيانات
- النوع بيتحدد تلقائي حسب القيمة

```
let x = 10;      // Number
x = "Hello";    // String
```

أنواع البيانات الأساسية في JavaScript

Primitive Data Types (أنواع بسيطة)

1. Number

2. String

3. Boolean

4. Undefined

5. Null

6. Symbol

7. BigInt

Number 1

أي رقم:

- صحيح
- عشري
- سالب أو موجب

```
let age = 25;
```

```
let price = 99.99;  
let temperature = -5;
```

ملاحظات:

```
typeof 10; // "number"  
typeof 3.14; // "number"
```

String 2

نصوص، بتتحت بين:

" " •

' ' •

(Template literals) •

```
let name = "Youssef";  
let city = 'Cairo';  
let message = `Hello ${name}`;
```

Boolean 3

قيم منطقية:

- true

- false

```
let isLoggedIn = true;  
let hasPermission = false;
```

غالبًا يستخدم في:

- الشروط

- التحكم في السلوك

Undefined

يعني:

- المتغير تعرّف

- لكن ماخذه قيمة

```
let x;  
console.log(x); // undefined
```

JavaScript بتخطيها تلقائيًا.

Null 5

يعني:

- قيمة فاضية
- أنت محددها بإيدك

```
let user = null;
```

الفرق:

- undefined → JS
- null → المطور

Symbol 6 (متقدم شوية)

يستخدم لإنشاء قيمة فريدة:

```
const id = Symbol("id");
```

استخدامه قليل في المستوى المبتدىء.

BigInt 7

للأرقام الكبيرة جدًا:

```
const bigNumber = 12345678901234567890n;
```

(Non-Primitive (Reference Types

Object 1

```
let user = {  
  name: "Ali",  
  age: 30  
};
```

Array 2

```
let colors = ["red", "green", "blue"];
```

Function 3

```
function sayHello() {  
  console.log("Hello");  
}
```

كلهم نوعهم:

```
typeof {}      // "object"  
typeof []      // "object"  
typeof function(){} // "function"
```


الفرق بين Reference و Primitive

:Primitive

- يتخزن بالقيمة
- نسخة مستقلة

```
let a = 5;  
let b = a;  
b = 10;
```

```
console.log(a); // 5
```

:Reference

- يتخزن بالمرجع

```
let obj1 = { name: "Ali" };  
let obj2 = obj1;
```

```
obj2.name = "Ahmed";
```

```
console.log(obj1.name); // Ahmed
```

typeof Operator

```
typeof 10;           // "number"  
typeof "Hi";         // "string"  
typeof true;         // "boolean"  
typeof undefined;    // "undefined"  
typeof null;         // "object" ❌ (Bug قديم)
```

أخطاء شائعة

- الخلط بين `null` و `undefined`
- توقع إن `Array` نوعه `array`
- مقارنة `Object` بـ `Object` باستخدام `==`
- الاعتماد على `typeof null`

اختر الإجابة الصحيحة (MCQ)

1 أي نوع بيانات للنصوص؟

- A) Number
- B) Boolean
- C) String
- D) Object

2 قيمة لم يتم إعطاؤها بعد؟

- A) null
- B) undefined
- C) false
- D) 0

3) typeof [] يرجع؟

A) array

B) object

C) list

D) undefined

What is a Data Type?

A data type determines:

- What is this value?
- What can we do with it?

Example:

```
10          // Number
```

```
"Hello"     // Text
```

```
true       // Boolean
```

JavaScript is a Dynamic Language

This means:

- You don't need to specify the data type
- Type is determined automatically based on the value

```
let x = 10;      // Number
```

```
x = "Hello";    // String
```

Basic Data Types in JavaScript

Primitive Data Types (Simple Types)

- Number
- String
- Boolean
- Undefined
- Null
- Symbol
- BigInt

1 Number

Any number:

- Integer
- Decimal
- Negative or Positive

```
let age = 25;  
let price = 99.99;  
let temperature = -5;
```

Notes:

```
typeof 10; // "number"  
typeof 3.14; // "number"
```

2 String

Texts, put between:

- " "

- `' '`
- (Template literals)

```
let name = "Youssef";  
let city = 'Cairo';  
let message = `Hello ${name}`;
```

3 Boolean

Logical values:

- `true`
- `false`

```
let isLoggedIn = true;  
let hasPermission = false;
```

Usually used in:

- Conditions
- Controlling behavior

4 Undefined

Means:

- Variable is declared
- But has no value yet

```
let x;  
console.log(x); // undefined
```

JavaScript assigns it automatically.

5 Null

Means:

- Empty value
- You set it manually

```
let user = null;
```

Difference:

- undefined → JS
- null → Developer

6 Symbol (Advanced)

Used to create a unique value:

```
const id = Symbol("id");
```

Used rarely for beginners.

7 BigInt

For very big numbers:

```
const bigNumber = 12345678901234567890n;
```

Non-Primitive (Reference Types)

1 Object

```
let user = {  
  name: "Ali",  
  age: 30  
};
```

2 Array

```
let colors = ["red", "green", "blue"];
```

3 Function

```
function sayHello() {
```

```
    console.log("Hello");  
}
```

All their types:

```
typeof {}      // "object"  
typeof []     // "object"  
typeof function(){} // "function"
```

Difference between Primitive & Reference

Primitive:

- Stored by value
- Independent copy

```
let a = 5;  
let b = a;  
b = 10;
```

```
console.log(a); // 5
```

Reference:

- Stored by reference

```
let obj1 = { name: "Ali" };
```

```
let obj2 = obj1;
```

```
obj2.name = "Ahmed";
```

```
console.log(obj1.name); // Ahmed
```

typeof Operator

```
typeof 10;           // "number"
```

```
typeof "Hi";         // "string"
```

```
typeof true;         // "boolean"
```

```
typeof undefined;    // "undefined"
```

```
typeof null;           // "object" ❌ (Old bug)
```

Common Mistakes

- Confusing null and undefined
- Expecting Array type to be "array"
- Comparing Object with Object using ==
- Relying on `typeof null`

Choose the correct answer (MCQ)

1) Which data type is for text?

- A) Number
- B) Boolean
- C) String
- D) Object

2) Value that is not assigned yet?

- A) null
- B) undefined
- C) false
- D) 0

3) `typeof [ ]` returns?

- A) array
- B) object
- C) list
- D) undefined

Truthy & Falsy

Truthy & Falsy

يعني إيه Truthy و Falsy؟

في JavaScript أي قيمة:

- يا إما تعتبر **true**

- يا إما تعتبر **false**

حتى لو مش **true** أو **false** صراحة

ده بيظهر جدًا في:

- **if**

- **while**

- **&&** و **||**

القيم الـ Falsy (عدددهم قليل)

القيم دي دايماً **false**:

false

0


```
-0  
0n  
""  
null  
undefined  
NaN
```

Truthy = أي حاجة غير دول

Truthy أمثلة 

```
"hello"  
1  
-5  
[]  
{}  
"0"  
"false"
```

حتى Array فاضية و Object فاضي = Truthy

أمثلة عملية

مثال 1 `if statement`

```
const name = "";

if (name) {
  console.log("Hello " + name);
} else {
  console.log("No name provided");
}
```

✔ هيطبع: No name provided

مثال 2 `تحقق من داتا`

```
if (users.length) {
  renderUsers();
}
```

لو Array فاضية $\rightarrow$ Falsy $\rightarrow$ length = 0

مثال 3 `استخدام &&`

```
isLoggedIn && showDashboard();
```

لو `isLoggedIn = false`

الدالة مش هتتنفذ

مثال 4 OR ||

```
const username = inputValue || "Guest";
```

لو `inputValue` فاضي

هستخدم "Guest"

Bugs شائعة جدًا

0 يعتبر Falsy ❌

"" يعتبر Falsy ❌

[] مش Falsy ❌

{ } مش Falsy ❌

```
if (price) {  
  applyDiscount();  
}
```

لو `price = 0`

الكود مش هيشغل

حل أنصف

```
if (price !== null && price !== undefined) {  
  applyDiscount();  
}
```

أو:

```
if (typeof price === "number") {  
  applyDiscount();  
}
```

Boolean Conversion

```
Boolean("")  
// false
```

```
!!"hello"
```

```
// true
```

Task صغير

المطلوب:

عندك Form Login

المطلوب تتحقق من البيانات باستخدام Truthy / Falsy

الشروط:

1. لو الإيميل أو الباسورد فاضي

→ اطبع "Please fill all fields"

2. لو الاثنين موجودين

→ اطبع "Login success"

3. استخدم:

○ Truthy / Falsy

○ من غير مقارنة مباشرة بـ ""

What is Truthy & Falsy?

In JavaScript, any value:

- Either considered **true**
- Or considered **false**

Even if it is not literally **true** or **false**.

This appears a lot in:

- **if**
- **while**
- **&&** and **||**

Falsy values (few in number)

These values are always false:

- false

- 0

- -0

- 0n

- ""

- null

- undefined

- NaN

Anything else = **Truthy**

Truthy Examples

"hello"

1

-5

[]

{}

"0"

"false"

Even empty Array [] and empty Object {} = Truthy

Practical Examples

Example 1 if statement

```
const name = "";
```

```
if (name) {
```

```
  console.log("Hello " + name);
```



```
} else {  
  
  console.log("No name provided");  
  
}
```

✓ Output: No name provided

Example 2 Check data

```
if (users.length) {  
  
  renderUsers();  
  
}
```

- If Array is empty → length = 0 → Falsy

Example 3 Using &&

```
isLoggedIn && showDashboard();
```

- If `isLoggedIn = false` → function will not run

Example 4 Using OR ||

```
const username = inputValue || "Guest";
```

- If `inputValue` is empty → "Guest" will be used

Very common bugs

- ❌ `0` is Falsy
- ❌ `""` is Falsy
- ❌ `[]` is **not** Falsy

- **✗** {} is **not** Falsy

```
if (price) {  
  
    applyDiscount();  
  
}
```

- If price = 0 → code will not run

Cleaner solution

```
if (price !== null && price !== undefined) {  
  
    applyDiscount();  
  
}
```

Or:

```
if (typeof price === "number") {  
  
    applyDiscount();  
  
}
```

```
}
```

Boolean Conversion

```
Boolean(""); // false
```

```
!!"hello" // true
```

Small Task

Requirement:

- You have a Login Form
- Validate data using **Truthy / Falsy**

Conditions:

- If email or password is empty → print "Please fill all fields"
- If both exist → print "Login success"

Use **Truthy / Falsy**

- Without comparing directly to ""

Type coercion

Type coercion

يعني إيه Type Coercion؟

Type Coercion =

JavaScript بتحوّل نوع البيانات تلقائي من نوع لنوع عشان العملية تمشي.

JS بتحاول "تساعدك"...

أحيانًا بتضبط

وأحيانًا بتبوّظ الدنيا

إمتى بيحصل؟

بيحصل في:

- العمليات الحسابية

- المقارنات

- `if` و `while`

- `==`

Coercion مع العمليات

مثال (1) الجمع

```
1 + "5"
```

اللي بيحصل:

- JS تلاقي String
- تحوّل كله String

النتيجة: 

```
"51"
```

مثال (2) الطرح

```
1 - "5"
```

هنا:

- مفيش جمع String
- JS تحوّل "5" → Number

النتيجة: 

أمثلة سريعة

```
"10" * 2    // 20
"10" / 2     // 5
"10" - "2"   // 8
```

Coercion مع المقارنة

== (ببمعمل Coercion)

```
"5" == 5    // true
null == undefined // true
```

خطر جدًا ❌

=== (مفيش Coercion)

```
"5" === 5    // false
```

الأفضل دائمًا ✅

Boolean Coercion

```
if ("")      // false
if ("hello") // true
if (0)       // false
if ([])      // true
```


ربط مباشر مع Truthy / Falsy

أمثلة شهيرة محيرة

```
[] + [] // ""  
[] + {} // "[object Object]"  
{ } + [] // 0 (في بعض الحالات)
```

ليه؟

لأن JS بتحوّلهم لـ String أو Number حسب السياق

❌ أخطاء شائعة

- الاعتماد على ==
- استخدام + مع Strings من غير قصد
- افتراض إن JS مش هتحوّل القيم

Best Practices ✓

- ✓ استخدم ===
- ✓ اعمل Conversion بنفسك:

```
Number("5")  
String(10)  
Boolean(value)
```

✓ خلي الكود واضح مش ذكي زيادة

Task صغير

المطلوب:

عندك القيم دي:

```
const a = "10";  
const b = 5;
```

الشروط:

- 1 طبع ناتج جمعهم **رقم**
- 2 طبع ناتج جمعهم **كنص**
- 3 منع Type Coercion التلقائي

What is Type Coercion?

Type Coercion =

JavaScript automatically converts a value from one type to another so the operation can work.

- JS tries to “help you”...
- Sometimes it works 
- Sometimes it breaks things 

When does it happen?

- Arithmetic operations
- Comparisons
- `if` and `while`
- `==`

Coercion with Operations

Example 1 Addition

"5" + 1

What happens:

- JS sees a String
- Converts everything to String

✓ Result: "51"

Example 2 Subtraction

"5" - 1

Here:

- No String addition
- JS converts "5" → Number
✓ Result: 4

Quick examples:

```
"10" * 2    // 20
```

```
"10" / 2    // 5
```

```
"10" - "2"  // 8
```

Coercion with Comparisons

`==` (does Coercion)

```
"5" == 5    // true
```

```
null == undefined // true
```

✗ Very dangerous

`===` (no Coercion)

```
"5" === 5 // false
```

✓ Always preferred

Boolean Coercion

```
if ( "") // false
```

```
if ("hello") // true
```

```
if (0) // false
```

```
if ([]) // true
```

- Directly related to **Truthy / Falsy**

Famous confusing examples

```
[] + [] // ""
```

```
[] + {} // "[object Object]"
```

```
{ } + [] // 0 (in some cases)
```

Why?

- JS converts them to String or Number depending on context

Common Mistakes

- Relying on `==`
- Using `+` with Strings unintentionally
- Assuming JS won't convert values

Best Practices

- Use `===`
- Convert values yourself:

```
Number("5")
```

```
String(10)
```

```
Boolean(value)
```

- Keep code clear, not “too smart”

Small Task

Requirement:

You have these values:

```
const a = "10";
```

```
const b = 5;
```

Conditions:

- 1 Print their sum as a **number**
- 2 Print their sum as a **string**
- 3 Prevent automatic Type Coercion

Operators

Operators in JavaScript

JavaScript في (Operators) العمليات

يعني إيه Operator؟

Operator هو رمز بنستخدمه علشان نعمل عملية على قيمة أو أكثر.

مثال:

```
10 + 5    // +  
x = 10    // =  
x > 5     // >
```

أنواع الـ Operators في JavaScript

- 1 Arithmetic Operators
- 2 Assignment Operators
- 3 Comparison Operators
- 4 Logical Operators
- 5 Unary Operators
- 6 Ternary Operator

1 Arithmetic Operators (عمليات حسابية)

العملية	الرمز	مثال
جمع	+	3 + 5
طرح	-	4 - 10
ضرب	*	3 * 2
قسمة	/	2 / 10
باقي القسمة	%	3 % 10
أس	**	3 ** 2

مثال:

```
let a = 10;  
let b = 3;
```

```
console.log(a + b); // 13  
console.log(a % b); // 1
```

2 Assignment Operators (الإسناد)

```
let x = 10;
```

```
x += 5; // x = x + 5  
x -= 2; // x = x - 2
```

```
x *= 3; // x = x * 3
x /= 2; // x = x / 2
```

3 Comparison Operators (المقارنة)

Boolean (true / false) بترجع

الرمز	العملية
==	يساوي
===	يساوي بالقيمة والنوع
!=	لا يساوي
!==	لا يساوي بالنوع
<	أكبر من
>	أصغر من
<=	أكبر أو يساوي
>=	أصغر أو يساوي

مثال مهم:

```
5 == "5" // true
5 === "5" // false
```

:Best Practice

استخدم دائمًا ===

4 Logical Operators (منطقية)

الرمز	العملية
&&	AND
	OR
!	NOT

مثال:

```
let age = 20;  
let hasID = true;
```

```
age >= 18 && hasID; // true
```

5 Unary Operators (عملية على عنصر واحد)

```
let x = 5;
```

```
x++; // 6  
x--; // 5
```

:typeof

```
typeof "Hello"; // "string"
```

6 Ternary Operator (اختصار if)

```
condition ? valueIfTrue : valueIfFalse;
```

مثال:

```
let age = 17;
```

```
let result = age >= 18 ? "Adult" : "Minor";
```

Operator Precedence (الأولوية)

```
let result = 5 + 3 * 2; // 11
```

الضرب قبل الجمع

⚠️ أخطاء شائعة

- استخدام **==** بدل **===**
- نسيان أولوية العمليات
- الخلط بين **&&** و **||**
- استخدام **+** مع Strings بدون قصد

```
"5" + 5 // "55"
```

اختر الإجابة الصحيحة (MCQ)

1) ناتج الكود؟

```
5 + 3 * 2
```

- A) 16
- B) 11
- C) 10
- D) 13

2) أي Operator للمقارنة بالنوع والقيمة؟

- A) ==
- B) !=
- C) ===
- D) <=

3) ناتج؟

```
true && false
```

A) true

B) false

What is an Operator?

An operator is a symbol we use to perform an operation on one or more values.

Example:

10 + 5 // +

x = 10 // =

x > 5 // >

Types of Operators in JavaScript

- ① Arithmetic Operators
- ② Assignment Operators
- ③ Comparison Operators
- ④ Logical Operators
- ⑤ Unary Operators
- ⑥ Ternary Operator

① Arithmetic Operators

Operation	Symbol	Example
Addition	+	5 + 3
Subtraction	-	10 - 4
Multiplication	*	2 * 3
Division	/	10 / 2
Modulus (Remainder)	%	10 % 3
Exponent	**	2 ** 3

Example:

```
let a = 10;
```

```
let b = 3;
```

```
console.log(a + b); // 13
```

```
console.log(a % b); // 1
```

2 Assignment Operators

```
let x = 10;
```

```
x += 5; // x = x + 5
```

```
x -= 2; // x = x - 2
```

```
x *= 3; // x = x * 3
```

```
x /= 2; // x = x / 2
```

3 Comparison Operators

- Return Boolean (**true** / **false**)

Operation	Symbol
-----------	--------

Equal	==
Equal value & type	===
Not equal	!=
Not equal type & value	!==
Greater than	>
Less than	<
Greater or equal	>=
Less or equal	<=

Important example:

```
5 == "5"    // true
```

```
5 === "5"   // false
```

Best Practice: Always use `===`

4 Logical Operators

Operation	Symbol
AND	<code>&&</code>
OR	<code> </code>
NOT	<code>!</code>

Example:

```
let age = 20;
```

```
let hasID = true;
```

```
age >= 18 && hasID; // true
```

5) **Unary Operators** (operate on one value)

```
let x = 5;
```

```
x++; // 6
```

```
x--; // 5
```

```
typeof "Hello"; // "string"
```

6) **Ternary Operator** (short if)

```
condition ? valueIfTrue : valueIfFalse;
```

Example:

```
let age = 17;
```

```
let result = age >= 18 ? "Adult" : "Minor";
```

Operator Precedence (Order)

```
let result = 5 + 3 * 2; // 11
```

- Multiplication before addition

Common Mistakes

- Using `==` instead of `===`
- Forgetting operator precedence
- Confusing `&&` and `||`
- Using `+` with Strings unintentionally

```
"5" + 5 // "55"
```

Choose the correct answer (MCQ)

1 Result of the code?

`5 + 3 * 2`

A) 16

B) 11

C) 10

D) 13

2 Which operator compares **value & type**?

A) ==

B) !=

C) ===

D) <=

3 Result?

`true && false`

A) true

B) false

Conditions

Conditions in JavaScript

JavaScript الشروط (if / else / switch)

يعني إيه Condition؟

Condition = شرط

بناءً عليه JavaScript تقرر:

- تنفذ كود
- أو تتجاهله

تشبيه بسيط:

لو الجو حر → شغل التكييف

غير كده → افتح الشباك

if Statement **1**

```
if (condition) {  
    // code runs if condition is true  
}
```

مثال:

```
let age = 20;

if (age >= 18) {
  console.log("Allowed");
}
```

if / else 2

```
if (condition) {
  // true
} else {
  // false
}
```

مثال:

```
let isLoggedIn = false;

if (isLoggedIn) {
  console.log("Welcome");
} else {
  console.log("Please login");
}
```

3 else if (أكثر من شرط)

```
if (condition1) {  
} else if (condition2) {  
} else {  
}
```

مثال:

```
let score = 75;  
  
if (score >= 90) {  
  console.log("Excellent");  
} else if (score >= 70) {  
  console.log("Good");  
} else {  
  console.log("Try again");  
}
```

Comparison + Logical Operators مع الشروط

```
let age = 25;  
let hasID = true;  
  
if (age >= 18 && hasID) {  
  console.log("Access granted");  
}
```

Truthy & Falsy (مهم جدًا)

قيم بتتسبب false:

```
false  
0  
"  
null  
undefined  
NaN
```

أي حاجة غير كده = true

```
if ("hello") {  
  console.log("Runs"); //   
}
```

switch Statement

بدل لـ **if else** لما يكون عندك حالات ثابتة.

```
switch (value) {  
  case x:  
    break;  
  case y:  
    break;  
  default:  
}
```

مثال:

```
let day = "Monday";

switch (day) {
  case "Monday":
    console.log("Start of week");
    break;
  case "Friday":
    console.log("Weekend soon");
    break;
  default:
    console.log("Normal day");
}
```

break مهم جدًا عشان يمنع تنفيذ باقي الحالات.

5 Ternary Operator (مراجعة سريعة)

```
condition ? trueValue : falseValue;
```

مثال:

```
let age = 16;
let result = age >= 18 ? "Adult" : "Minor";
```

Best Practices

- استخدم `===`
- خلي الشروط بسيطة وواضحة
- بلاش تعقّد `if else` زيادة عن اللزوم
- استخدم `switch` لما يكون أنسب

أخطاء شائعة

- نسيان `break` في `switch`
- استخدام `==`
- الاعتماد على Truthy / Falsy بدون وعي
- شروط طويلة ومعقّدة

اختر الإجابة الصحيحة (MCQ)

1) ناتج الكود؟

```
if (0) {  
  console.log("Hello");  
} else {  
  console.log("Hi");  
}
```


A) Hello

B) Hi

2 أفضل للمقارنة؟

A) ==

B) ===

3 أي قيمة تعتبر Falsy؟

A) "0"

B) []

C) {}

D) null

What is a Condition?

Condition = a rule

- Based on it, JavaScript decides:
 - Run some code
 - Or ignore it

Simple example:

- If the weather is hot → turn on the AC
- Otherwise → open the window

1 if Statement

```
if (condition) {  
    // code runs if condition is true  
}
```

Example:

```
let age = 20;

if (age >= 18) {
  console.log("Allowed");
}
```

2 if / else

```
if (condition) {
  // true
} else {
  // false
}
```

Example:

```
let isLoggedIn = false;

if (isLoggedIn) {
  console.log("Welcome");
} else {
  console.log("Please login");
}
```

3 else if (multiple conditions)

```
if (condition1) {  
  } else if (condition2) {  
  } else {  
  }
```

Example:

```
let score = 75;  
  
if (score >= 90) {  
  console.log("Excellent");  
} else if (score >= 70) {  
  console.log("Good");  
} else {  
  console.log("Try again");  
}
```

Comparison + Logical Operators with Conditions

```
let age = 25;  
let hasID = true;  
  
if (age >= 18 && hasID) {  
  console.log("Access granted");  
}
```

Truthy & Falsy (Very important)

Values counted as false:

```
false
0
""
null
undefined
NaN
```

Anything else = true

```
if ("hello") {
  console.log("Runs"); // 
}
```

switch Statement

- Alternative to if/else when you have fixed cases

```
switch (value) {
  case x:
```

```
    break;
case y:
    break;
default:
}
```

Example:

```
let day = "Monday";

switch (day) {
  case "Monday":
    console.log("Start of week");
    break;
  case "Friday":
    console.log("Weekend soon");
    break;
  default:
    console.log("Normal day");
}
```

- **break** is very important to prevent running other cases

5 Ternary Operator (Quick Review)

```
condition ? trueValue : falseValue;
```

Example:

```
let age = 16;  
let result = age >= 18 ? "Adult" : "Minor";
```

Best Practices

- Use `===`
- Keep conditions simple and clear
- Don't overcomplicate if/else
- Use switch when it's more suitable

Common Mistakes

- Forgetting **break** in switch
- Using **==**
- Relying on Truthy/Falsy without awareness
- Long and complex conditions

Choose the correct answer (MCQ)

1) Result of the code?

```
if (0) {  
    console.log("Hello");  
} else {  
    console.log("Hi");  
}
```

A) Hello

B) Hi

2) Best for comparison?

A) ==

B) ===

3) Which value is Falsy?

A) "0"

B) []

C) {}

D) null

Loops

Loops in JavaScript

JavaScript الحلقات التكرارية في

يعني إيه Loop؟

Loop = تكرار

بدل ما تكتب نفس الكود 10 مرات، تخليه يتكرر تلقائيًا.

تشبيه بسيط:

عندك لسته أسماء

بدل ما تطبع كل اسم بإيدك

تقول للكمبيوتر: لف عليهم واحد واحد

أنواع الـ Loops في JavaScript

1 for

2 while

3 do...while

4 for...of

5 for...in

1 for Loop (الأشهر)

```
for (start; condition; increment) {  
  // code  
}
```

مثال:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

الاستخدام:

- لما تكون عارف عدد التكرارات

2 while Loop

```
while (condition) {  
  // code  
}
```

مثال:

```
let i = 0;

while (i < 3) {
  console.log(i);
  i++;
}
```

الاستخدام:

- لما الشرط أهم من العدد

do...while Loop **3**

```
do {
  // code
} while (condition);
```

مثال:

```
let x = 5;

do {
  console.log(x);
} while (x < 3);
```

يُنفذ مرة واحدة على الأقل.

4 for...of (للتعامل مع Arrays)

```
for (let item of array) {  
  // code  
}
```

مثال:

```
let colors = ["red", "green", "blue"];  
  
for (let color of colors) {  
  console.log(color);  
}
```

أفضل للقراءة من **for** التقليدي.

5 for...in (للتعامل مع Objects)

```
for (let key in object) {  
  // object[key]  
}
```

مثال:

```
let user = {  
  name: "Ali",  
  age: 25  
};
```

```
for (let key in user) {  
  console.log(key, user[key]);  
}
```

break & continue

break

يوقف اللوب:

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) break;  
  console.log(i);  
}
```

continue

يتخطى لفة:

```
for (let i = 0; i < 5; i++) {  
  if (i === 2) continue;  
  console.log(i);  
}
```

Arrays مع Loops

```
let numbers = [1, 2, 3];  
  
for (let i = 0; i < numbers.length; i++) {  
  console.log(numbers[i]);  
}
```

متى تستخدم كل Loop؟

الأفضل	الحالة
for	عدد معروف
while	شرط فقط
do...while	لازم يتنفذ مرة
for...of	Array
for...in	Object

أخطاء شائعة

-  Infinite Loop → نسيان زيادة العداد
- استخدام `for...in` مع Array
- استخدام `var` بدل `let`
- التعديل على Array داخل Loop بدون وعي

اختر الإجابة الصحيحة (MCQ)

1 أي Loop مناسب لـ Array؟

- A) for...in
- B) for...of
- C) while
- D) do...while

2 أي Loop يتنفذ مرة واحدة على الأقل؟

- A) for
- B) while
- C) do...while
- D) for...of

3 نتائج؟

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

A) 1 2 3

B) 0 1 2

C) 0 1 2 3

D) 1 2

What is a Loop?

Loop = repetition

- Instead of writing the same code 10 times, you let it repeat automatically.

Simple example:

- You have a list of names
- Instead of printing each name manually
- You tell the computer: go through them one by one

Types of Loops in JavaScript

1 for

2 while

3 do...while

4 for...of

5 for...in

1 for Loop (most common)

```
for (start; condition; increment) {  
  // code  
}
```

Example:

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

Use case: when you know the number of repetitions

2 while Loop

```
while (condition) {  
  // code  
}
```

Example:

```
let i = 0;  
  
while (i < 3) {
```

```
    console.log(i);  
    i++;  
}
```

Use case: when the condition is more important than the count

3 do...while Loop

```
do {  
    // code  
} while (condition);
```

Example:

```
let x = 5;  
  
do {  
    console.log(x);  
} while (x < 3);
```

- Executes **at least once**

4 for...of (for Arrays)

```
for (let item of array) {  
  // code  
}
```

Example:

```
let colors = ["red", "green", "blue"];  
  
for (let color of colors) {  
  console.log(color);  
}
```

- Better readability than traditional `for`

5 for...in (for Objects)

```
for (let key in object) {  
  // object[key]  
}
```

Example:

```
let user = {  
  name: "Ali",  
  age: 25  
};  
  
for (let key in user) {  
  console.log(key, user[key]);  
}
```

break & continue

break – stops the loop:

```
for (let i = 0; i < 5; i++) {  
  if (i === 3) break;  
  console.log(i);  
}
```

continue – skips an iteration:

```
for (let i = 0; i < 5; i++) {  
  if (i === 2) continue;  
  console.log(i);  
}
```

Loops with Arrays

```
let numbers = [1, 2, 3];

for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}
```

When to use each Loop?

Case	Best Loop
Known number	for
Condition only	while
Must run once	do...while
Array	for...of
Object	for...in

Common Mistakes

- Forgetting to increase the counter → Infinite Loop ❌
- Using `for...in` with Array
- Using `var` instead of `let`
- Modifying an Array inside a Loop without awareness

Choose the correct answer (MCQ)

❏ Which Loop is suitable for an Array?

- A) `for...in`
- B) `for...of`
- C) `while`
- D) `do...while`

❏ Which Loop runs at least once?

- A) `for`
- B) `while`
- C) `do...while`
- D) `for...of`

3 Result?

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

A) 1 2 3

B) 0 1 2

C) 0 1 2 3

D) 1 2

Functions

Functions in JavaScript

JavaScript في (Functions) الدوال

يعني إيه Function؟

Function = مجموعة أوامر

بتنفذ لما نستدعيها، وبتساعدنا:

- نعيد استخدام الكود
- ننظم البرنامج
- نقلل التكرار

تشبيه بسيط:

Function = ماكينة

تدخلها بيانات → تطلع نتيجة

ليه بنستخدم Functions؟

- إعادة الاستخدام
- تقسيم البرنامج لأجزاء
- كود أوضح وأسهل صيانة

1 Function Declaration (الشكل التقليدي)

```
function sayHello() {  
  console.log("Hello!");  
}
```

استدعاء:

```
sayHello();
```

2 Parameters & Arguments

```
function greet(name) {  
  console.log("Hello " + name);  
}
```

```
greet("Youssef");
```

- **Parameter**: المتغير داخل الفنكشن

- **Argument:** القيمة التي بنبعتها

Return Statement 3

```
function sum(a, b) {  
  return a + b;  
}
```

```
let result = sum(5, 3);
```

بعد **return** الكود يوقف.

Function Expression 4

```
const multiply = function (a, b) {  
  return a * b;  
};
```

الفنكشن هنا متخزنة في متغير.

(Arrow Functions (ES6 5

```
const add = (a, b) => {  
  return a + b;  
};
```

اختصار:

```
const add = (a, b) => a + b;
```

:Arrow Functions

- أبسط
- مستخدمة بكثرة في React

Default Parameters

```
function greet(name = "Guest") {  
  console.log("Hello " + name);  
}
```

```
greet();
```

Scope داخل Functions

```
let x = 10;
```

```
function test() {  
  let y = 5;  
  console.log(x); //   
}
```

```
console.log(y); // 
```

Callback Functions (مهم جدًا)

```
function process(callback) {  
  callback();  
}
```

```
process(function () {  
  console.log("Done");  
});
```

يستخدم كثير مع:

- Events
- Arrays
- Async code

Arrays مع Functions

```
let numbers = [1, 2, 3];
```

```
numbers.forEach(function (num) {  
  console.log(num);  
});
```


Best Practices

- اسم الفنكشن يكون واضح
- الفنكشن تعمل حاجة واحدة
- استخدم Arrow Functions عند الحاجة
- بلاش تعتمد على Global Variables

أخطاء شائعة

- نسيان `return`
- استخدام متغيرات خارج الفنكشن بدون قصد
- تعريف فنكشن داخل Loop بدون داعي
- سوء استخدام Arrow Functions

اختر الإجابة الصحيحة (MCQ)

1 وظيفة الفنكشن؟

- (A) تخزين بيانات
- (B) تكرار كود
- (C) تنظيم وإعادة استخدام الكود
- (D) رسم الصفحة

2 الفرق بين Argument و Parameter؟

- (A) مفيش فرق
- (B) Parameter داخل الفنكشن
- (C) Argument داخل الفنكشن
- (D) الاثنين نفس الحاجة

3 ناتج؟

```
function test() {  
  return;  
  console.log("Hi");  
}
```

- A) Hi
- B) undefined
- C) Error
- D) null

What is a Function?

Function = a set of commands

- Runs when we call it, and helps us:
 - Reuse code
 - Organize the program
 - Reduce repetition

Simple example:

- Function = machine
- You give it data → it gives a result

Why use Functions?

- Reuse code

- Split program into parts
- Clearer and easier to maintain

1 Function Declaration (classic way)

```
function sayHello() {  
    console.log("Hello!");  
}
```

Call it:

```
sayHello();
```

2 Parameters & Arguments

```
function greet(name) {  
    console.log("Hello " + name);  
}
```

```
greet("Youssef");
```

- **Parameter:** variable inside the function
- **Argument:** value we pass

3 Return Statement

```
function sum(a, b) {  
    return a + b;  
}
```

```
let result = sum(5, 3);
```

- After **return** the function stops

4 Function Expression

```
const multiply = function (a, b) {  
  return a * b;  
};
```

- Function stored in a variable

5 Arrow Functions (ES6)

```
const add = (a, b) => {  
  return a + b;  
};
```

- Shorter version:

```
const add = (a, b) => a + b;
```

- Arrow Functions: simple and widely used in React

Default Parameters

```
function greet(name = "Guest") {  
  console.log("Hello " + name);  
}
```

```
greet();
```

Scope inside Functions

```
let x = 10;
```

```
function test() {  
  let y = 5;  
  console.log(x); //   
}
```

```
console.log(y); // ❌
```

Callback Functions (very important)

```
function process(callback) {  
    callback();  
}
```

```
process(function () {  
    console.log("Done");  
});
```

- Used a lot with:

- Events

- Arrays

- Async code

Functions with Arrays

```
let numbers = [1, 2, 3];  
  
numbers.forEach(function (num) {  
    console.log(num);  
});
```

Best Practices

- Function name should be clear
- Function does one thing
- Use Arrow Functions when needed

- Avoid relying on Global Variables

Common Mistakes

- Forgetting `return`
- Using variables outside function by mistake
- Defining function inside a Loop without reason
- Misusing Arrow Functions

Choose the correct answer (MCQ)

1) Function's role?

- A) Store data
- B) Repeat code
- C) Organize and reuse code
- D) Draw the page

2) Difference between Parameter and Argument?

- A) No difference
- B) Parameter is inside function
- C) Argument is inside function
- D) Both are the same

3) Result?

```
function test() {  
    return;  
    console.log("Hi");  
}
```

- A) Hi
- B) undefined
- C) Error
- D) null

Arrow functions

Arrow functions

يعني إيه Arrow Function؟

هي طريقة **أقصر وأنصف** لكتابة الـ functions في JavaScript
واتضافت في ES6.

الشكل الأساسي

Function عادية

```
function sum(a, b) {  
  return a + b;  
}
```

Arrow Function

```
const sum = (a, b) => {  
  return a + b;  
};
```

نفس النتيجة، كود أقل

Short Syntax

لو الفنكشن:

- سطر واحد
- بترجع قيمة مباشرة

تقدر تشيل:

- `{ }`
- `return`

```
const sum = (a, b) => a + b;
```

Parameter واحد

```
const double = x => x * 2;
```

الأنقواس اختيارية لو Parameter واحد

مفیش Parameters

```
const sayHi = () => console.log("Hi");
```

Return Object

دي غلطة مشهورة

غلط: ❌

```
const user = () => { name: "Ali" };
```

صح: ✅

```
const user = () => ({ name: "Ali" });
```

الفرق المهم: **this**

Function عادية

- **this** بتحدد وقت التنفيذ

Arrow Function

- مفياش **this** خاص بيها
- بترث **this** من المكان اللي اكتبته فيه

مثال 1

```
const user = {  
  name: "Youssef",  
  sayHi: function () {  
    setTimeout(function () {  
      console.log(this.name);  
    }, 1000);  
  },  
};
```

✗ undefined

مثال 2 الحل بالـ Arrow

```
const user = {  
  name: "Youssef",  
  sayHi: function () {  
    setTimeout(() => {  
      console.log(this.name);  
    }, 1000);  
  },  
};
```

✓ "Youssef"

Arrow Functions ماينفعش في:

- Constructor functions
- Methods محتاجة **this** خاص بيها
- **arguments** object

```
const test = () => {  
  console.log(arguments); // ❌  
};
```

استخدامات شائعة

- Callbacks
- **map, filter, reduce**
- Event handlers (بحذر)

```
items.map(item => item.price);
```

Task صغير

المطلوب:

حوّل الكود ده لـ Arrow Functions:

```
function isEven(num) {  
  if (num % 2 === 0) {  
    return true;  
  } else {  
    return false;  
  }  
}
```

الشروط:

- استخدم Arrow Function
- استخدم Short Syntax
- رجع Boolean مباشرة

What is an Arrow Function?

- A shorter and cleaner way to write functions in JavaScript
- Added in ES6

Basic Syntax

Regular Function

```
function sum(a, b) {  
    return a + b;  
}
```

Arrow Function

```
const sum = (a, b) => {  
    return a + b;  
};
```

- Same result, less code

Short Syntax

- If the function has **one line** and **returns a value directly**, you can remove `{ }` and `return`

```
const sum = (a, b) => a + b;
```

Single Parameter

```
const double = x => x * 2;
```

- Parentheses are optional if only one parameter

No Parameters

```
const sayHi = () => console.log("Hi");
```

Returning an Object

Common mistake:

 Wrong:

```
const user = () => { name: "Ali" };
```

 Correct:

```
const user = () => ({ name: "Ali" });
```

Important Difference: **this**

Regular Function

- **this** is set at runtime

Arrow Function

- No own `this`
- Inherits `this` from the place it was written

Example 1

```
const user = {  
  name: "Youssef",  
  sayHi: function () {  
    setTimeout(function () {  
      console.log(this.name);  
    }, 1000);  
  },  
};
```

✗ undefined

Example 2 – Fix with Arrow Function

```
const user = {
```

```
name: "Youssef",
sayHi: function () {
  setTimeout(() => {
    console.log(this.name);
  }, 1000);
},
};
```

✓ "Youssef"

Arrow Functions should NOT be used in:

- Constructor functions
- Methods that need their own `this`
- `arguments` object

```
const test = () => {
  console.log(arguments); // ✗
```

```
};
```

Common Uses

- Callbacks
- `map, filter, reduce`
- Event handlers (careful)

```
items.map(item => item.price);
```

Small Task

- Convert this code to Arrow Function:

```
function isEven(num) {  
  if (num % 2 === 0) {
```



```
        return true;
    } else {
        return false;
    }
}
```

Requirements:

- Use Arrow Function
- Use Short Syntax
- Return Boolean directly

Scope & Hoisting (Simplified)

Scope & Hoisting (Simplified)

النطاق ورفع التعريف في JavaScript

يعني إيه Scope؟

Scope = المكان اللي المتغير أو الفنكشن يقدر يعيش ويتشاف فيه.

بمعنى:

هل المتغير ده متاح هنا؟ ولا لأ؟

أنواع الـ Scope (ببساطة)

- 1 Global Scope
- 2 Function Scope
- 3 Block Scope

Global Scope 1

متغيرات متعرفة برا أي block أو function

```
let name = "Youssef";
```

```
function sayHi() {  
  console.log(name);  
}
```

أي مكان في الكود يقدر يشوفه.

Function Scope 2

متغيرات متعرفة جوه function

```
function test() {  
  let x = 10;  
}  
console.log(x); // ❌ Error
```

برّه الفنكشن مش شايفها.

3 Block Scope (مهم جدًا)

الـ block هو:

```
{  
  // block  
}
```

مع let و const:

```
if (true) {  
  let a = 5;  
  const b = 10;  
}
```

```
console.log(a); // ❌
```

مع var:

```
if (true) {  
  var c = 20;  
}
```

```
console.log(c); // ✅
```

وذلك سبب خطورة **var**.

ملخص Scope سريع

const	let	var	النوع
✓	✓	✓	Global
✓	✓	✓	Function
✓	✓	✗	Block

Hoisting (ببساطة)

يعني إيه Hoisting؟

JavaScript **Hoisting** بترفع التعريفات لفوق قبل التنفيذ.

مش معناها إن الكود بيتنقل

لكن JS بتجهز الذاكرة الأول.

var مع Hoisting

```
console.log(x);  
var x = 10;
```

اللي بيحصل فعليًا:

```
var x;  
console.log(x); // undefined  
x = 10;
```

const و let مع Hoisting

```
console.log(y);  
let y = 5; ❌ Error
```

لأنهم في حاجة اسمها:

Temporal Dead Zone

Functions مع Hoisting

(بيشتغل) Function Declaration

```
sayHello();
```

```
function sayHello() {  
  console.log("Hello");  
}
```

(١١) Function Expression

```
sayHi(); // ❌
```

```
const sayHi = function () {  
  console.log("Hi");  
};
```

ليه ده مهم؟

- تفهم ليه Errors بتحصل
- تكتب كود أنضف
- تتجنب Bugs غريبة

Best Practices

- استخدم **let** و **const**
- عرّف المتغير قبل استخدامه
- بلاش تعتمد على Hoisting
- خلي التعريفات فوق

أخطاء شائعة

- استخدام var
- استخدام متغير قبل تعريفه
- الاعتماد على Hoisting
- الخلط بين function declaration و expression

اختر الإجابة الصحيحة (MCQ)

1 أي نوع Scope خاص بـ let؟

- A) Global
- B) Function
- C) Block
- D) File

2 نتائج؟

```
console.log(x);  
var x = 5;
```

- A) 5
- B) undefined

C) Error

D) null

3 نتائج؟

```
console.log(y);  
let y = 10;
```

A) undefined

B) 10

C) Error

D) null

What is Scope?

Scope = The area where a variable or function can exist and be accessed.

In other words:

Can this variable be used here or not?

Types of Scope (Simple)

- 1 Global Scope
- 2 Function Scope
- 3 Block Scope

1 Global Scope

- Variables defined **outside any function or block**

```
let name = "Youssef";
```

```
function sayHi() {  
  console.log(name);  
}
```

- Any part of the code can access it

2 Function Scope

- Variables defined **inside a function**

```
function test() {  
  let x = 10;  
}  
console.log(x); //  Error
```

- Outside the function cannot see it

3 Block Scope (Very Important)

- A block is:

```
{  
  // block  
}
```

- With `let` and `const`:

```
if (true) {  
  let a = 5;  
  const b = 10;  
}
```

```
console.log(a); // ❌
```

- With `var`:

```
if (true) {  
  var c = 20;  
}
```

```
console.log(c); // ✅
```

- This is why `var` can be risky

Quick Scope Summary

Type	var	let	const
Global	✓	✓	✓
Function	✓	✓	✓
Block	✗	✓	✓

Hoisting (Simple)

- What is Hoisting?

Hoisting = JavaScript **moves definitions to the top** before running the code.

- It **does not move the code**, but JS prepares memory first

Hoisting with var

```
console.log(x);  
var x = 10;
```

What actually happens:

```
var x;  
console.log(x); // undefined  
x = 10;
```

Hoisting with let & const

```
console.log(y);  
let y = 5; // ❌ Error
```

- They are in the **Temporal Dead Zone**

Hoisting with Functions

Function Declaration (Works)

```
sayHello();  
  
function sayHello() {  
    console.log("Hello");  
}
```

Function Expression (Does NOT work)

```
sayHi(); // ❌
```

```
const sayHi = function () {  
  console.log("Hi");  
};
```

Why is this important?

- Understand why Errors happen
- Write cleaner code
- Avoid strange bugs

✅ Best Practices

- Use `let` and `const`
- Define variables before using them

- Don't rely on Hoisting
- Keep definitions at the top

Common Mistakes

- Using `var`
- Using a variable before defining it
- Relying on Hoisting
- Confusing function declaration and expression

Choose the correct answer (MCQ)

1 Which Scope is for `let`?

A) Global

B) Function

C) Block

D) File

2) Output?

```
console.log(x);  
var x = 5;
```

A) 5

B) undefined

C) Error

D) null

3) Output?

```
console.log(y);  
let y = 10;
```

A) undefined

B) 10

C) Error

D) null

this keyword

this keyword

يعني إيه **this**؟

this = المتغير الخاص بالسياق الحالي

يعني: إيه اللي الكود بيشتغل جواه دلوقتي؟

كل context في JS له **this** خاص بيه

1 Global Context (سياق عالمي)

في المتصفح:

```
console.log(this);
```

✓ هيطلع: Window object

في Node.js:

✓ هيكون global object

Inside a Function 2

Function عادية

```
function sayHi() {  
  console.log(this);  
}
```

```
sayHi();
```

- في strict mode → **undefined**
- غير strict → global object

Function داخل Object (Method)

```
const user = {  
  name: "Youssef",  
  greet: function () {  
    console.log(this.name);  
  },  
};
```

```
user.greet(); // "Youssef"
```

هنا `this = Object`  `user` → اللي استدعى الفنكشن

Arrow Function

```
const user = {  
  name: "Youssef",  
  greet: () => {  
    console.log(this.name);  
  },  
};
```

```
user.greet(); // undefined
```

Arrow Function  مفيهاش `this` خاص بيها

- بترث `this` من المكان اللي اكتبته فيه

Summary Table

<code>= this</code>	Context
---------------------	---------

window / global	Global
undefined	(Function (strict
Object نفسه	Object method
this من lexical scope	Arrow Function

Common Mistakes 4

❌ نسيان الفرق بين Function عادية و this → Arrow يبقى undefined

❌ استخدام this في Global scope في Node → مش Window

❌ توقع this يشتغل داخل setTimeout بدون arrow

مثال عملي: setTimeout

```
const user = {
  name: "Youssef",
  greet: function () {
    setTimeout(function () {
      console.log(this.name);
    }, 1000);
  },
}
```

```
};
```

```
user.greet(); // undefined
```

الحل: 

```
const user = {  
  name: "Youssef",  
  greet: function () {  
    setTimeout(() => {  
      console.log(this.name);  
    }, 1000);  
  },  
};
```

```
user.greet(); // "Youssef"
```

Task صغير

المطلوب:

اعمل Object اسمه `car`:

`year` و `properties`: `brand` 1

الـ `showInfo` method: `تطبع` 2

```
"{This car is a {brand}, made in {year}"
```

What is **this**?

this = A special variable for the **current context**

- Means: what is the code running **inside of right now**?
- Every context in JS has its own **this**

1 Global Context

- In Browser:

```
console.log(this);
```

✓ Prints: **Window** object

- In Node.js:
 - ✓ It's **global** object

2 Inside a Function

Regular Function

```
function sayHi() {  
  
  console.log(this);  
  
}
```

```
sayHi();
```

- In strict mode → `undefined`
- Not strict → global object

Function inside Object (Method)

```
const user = {  
  
  name: "Youssef",
```

```
greet: function () {  
  
  console.log(this.name);  
  
},  
  
};
```

```
user.greet(); // "Youssef"
```

✓ Here **this** = **the object that called the function** → **user**

Arrow Function

```
const user = {  
  
  name: "Youssef",  
  
  greet: () => {  
  
    console.log(this.name);  
  
  },  
  
};
```

```
};
```

```
user.greet(); // undefined
```

✗ Arrow Functions **do NOT** have their own **this**

- They inherit **this** from the place they were **defined (lexical scope)**

3 Summary Table

Context	this =
Global	window / global
Function (strict)	undefined

Object method	the object itself
Arrow Function	inherited from lexical scope

4 Common Mistakes

✗ Forgetting the difference between **regular function** & **arrow**

function → **this** can be **undefined**

✗ Using **this** in Global scope in Node.js → not **Window**

✗ Expecting **this** to work in **setTimeout** without arrow

Practical Example: setTimeout

Wrong:

```
const user = {  
  
  name: "Youssef",  
  
  greet: function () {  
  
    setTimeout(function () {
```

```
        console.log(this.name);

    }, 1000);

},

};
```

```
user.greet(); // undefined
```

✓ **Correct with Arrow Function:**

```
const user = {

    name: "Youssef",

    greet: function () {

        setTimeout(() => {

            console.log(this.name);

        }, 1000);

    },

};
```

```
};
```

```
user.greet(); // "Youssef"
```

Small Task

- Create an Object called `car`:
 - 1 Properties: `brand` and `year`
 - 2 Method: `showInfo` prints:

```
"This car is a {brand}, made in {year}"
```


Arrays & Objects

Arrays & Objects

JavaScript المصفوفات والكائنات في

أولاً: Arrays (المصفوفات)

يعني إيه Array؟

Array = مجموعة قيم متخزنة في متغير واحد، بترتيب.

كل عنصر ليه **index** (رقم) بيبدأ من 0

```
let fruits = ["Apple", "Banana", "Orange"];
```

الوصول لعناصر Array

```
fruits[0]; // "Apple"  
fruits[1]; // "Banana"
```

تعديل العناصر

```
fruits[1] = "Mango";
```

طول الـ Array

```
fruits.length; // 3
```

أهم Array Methods (أساسي جدًا)

إضافة عناصر

```
fruits.push("Grapes"); // آخر المصفوفة  
fruits.unshift("Kiwi"); // أول المصفوفة
```

حذف عناصر

```
fruits.pop(); // آخر عنصر  
fruits.shift(); // أول عنصر
```

Array على Loop

```
for (let i = 0; i < fruits.length; i++) {  
    console.log(fruits[i]);  
}
```

أو

```
fruits.forEach(function (item) {  
    console.log(item);  
});
```

Array تحتوي على أي نوع بيانات

```
let data = [1, "hello", true, null];
```

ثانيًا: Objects

يعني إيه Object؟

Object = مجموعة خصائص (key : value)

```
let user = {  
  name: "Ahmed",  
  age: 25,  
  isAdmin: false  
};
```

الوصول لخصائص Object

Dot notation

```
user.name;
```

Bracket notation

```
user["age"];
```

نستخدم brackets لما:

- الاسم متغير
- فيه مسافة

تعديل وإضافة خصائص

```
user.age = 26;  
user.city = "Cairo";
```

حذف خاصية

```
delete user.isAdmin;
```

Object فيه (Functions Methods)

```
let user = {  
  name: "Sara",  
  sayHi: function () {  
    console.log("Hello " + this.name);  
  }  
};
```

```
user.sayHi();
```

Object على Loop

```
for (let key in user) {  
  console.log(key, user[key]);  
}
```

Arrays of Objects (مهم جدًا جدًا)

ده الشكل اللي بييجي من الـ APIs

```
let users = [  
  { name: "Ali", age: 20 },  
  { name: "Mona", age: 22 },  
  { name: "Omar", age: 25 }  
];
```

مثال:

```
console.log(users[1].name); // Mona
```

الفرق بين Array و Object

Object	Array
غير مرتب	بترتيب

key نصي	index رقمي
entity	list

Best Practices

- استخدم Array للقوائم
- استخدم Object لتمثيل كيان
- سمي المتغيرات بأسماء واضحة
- خليك consistent في الشكل

أخطاء شائعة

- نسيان index يبدأ من 0
- استخدام dot بدل brackets غلط
- نسيان الفرق بين array و object
- محاولة loop على object بـ for العادي

اختر الإجابة الصحيحة (MCQ)

1. أي نوع البيانات ده؟

```
let x = [ "HTML", "CSS", "JS" ];
```

- A) Object
- B) String
- C) Array
- D) Function

2. ناتج؟

```
let arr = [10, 20, 30];  
console.log(arr[1]);
```

- A) 10
- B) 20
- C) 30
- D) undefined

3. إزاي تضيف عنصر في آخر Array؟

- A) pop
- B) push

C) shift

D) unshift

What is an Array?

Array = A collection of values stored in **one variable**, in order.

- Each element has an **index** (number) starting from 0

```
let fruits = ["Apple", "Banana", "Orange"];
```

Access Array Elements

```
fruits[0]; // "Apple"
```

```
fruits[1]; // "Banana"
```

Modify Elements

```
fruits[1] = "Mango";
```

Array Length

```
fruits.length; // 3
```

Important Array Methods

Add elements:

```
fruits.push("Grapes"); // at the end
```

```
fruits.unshift("Kiwi"); // at the beginning
```

Remove elements:

```
fruits.pop(); // remove last
```

```
fruits.shift(); // remove first
```

Loop through Array

```
for (let i = 0; i < fruits.length; i++) {  
  console.log(fruits[i]);  
}
```

// OR

```
fruits.forEach(function(item) {  
  console.log(item);  
});
```

Arrays can store any data type

```
let data = [1, "hello", true, null];
```

Objects

What is an Object?

Object = A collection of **properties** (key : value)

```
let user = {  
  name: "Ahmed",  
  age: 25,  
  isAdmin: false  
};
```

Access Object Properties

- Dot notation:

```
user.name;
```

- Bracket notation:

```
user["age"];
```

Use brackets when:

- The key is **dynamic**
- The key has **spaces**

Modify & Add Properties

```
user.age = 26;
```

```
user.city = "Cairo";
```

Delete a property

```
delete user.isAdmin;
```

Objects can have Functions (Methods)

```
let user = {
```

```
name: "Sara",  
sayHi: function() {  
  console.log("Hello " + this.name);  
}  
};
```

```
user.sayHi();
```

Loop through Object

```
for (let key in user) {  
  console.log(key, user[key]);  
}
```

Arrays of Objects (Very Important!)

This is how data usually comes from APIs

```
let users = [  
  { name: "Ali", age: 20 },  
  { name: "Mona", age: 22 },
```

```
{ name: "Omar", age: 25 }  
];  
  
console.log(users[1].name); // "Mona"
```

Difference between Array & Object

Array	Object
Ordered	Unordered
Numeric index	Key as string
List	Entity

Best Practices

- Use **Array** for lists
- Use **Object** for representing entities

- Name variables **clearly**
- Be **consistent** in structure

Common Mistakes

- Forgetting index starts at 0
- Using dot instead of brackets incorrectly
- Confusing Array vs Object
- Trying to loop over Object with regular **for** loop

MCQs

1) What is the data type?

```
let x = ["HTML", "CSS", "JS"];
```

A) Object

B) String

C) Array ☒

D) Function

2) Output?

```
let arr = [10, 20, 30];
```

```
console.log(arr[1]);
```

A) 10

B) 20 ☒

C) 30

D) undefined

3) How to add an element at the **end** of an Array?

A) pop

B) push ☒

C) shift

D) unshift

Basic error types

Basic error types

يعني إيه Error؟

لما JS تواجه حاجة **مش مفهومة أو غلط** أثناء التنفيذ → بتطلع Error
الـ Error بيوقف الكود لو مش متعامل معاه.

1 Syntax Error (خطأ في الصياغة)

- بيحصل لما تكتب كود **مش مفهوم** لـ JS
- بيظهر قبل التنفيذ

مثال:

```
console.log("Hello"
```

هيديك: 

```
SyntaxError: missing ) after argument list
```

Reference Error (متغير مش معرف) 2

- يحصل لما تحاول تستخدم متغير مش موجود

مثال:

```
console.log(name);
```

هيديك: 

```
ReferenceError: name is not defined
```

Type Error (نوع غير متوقع) 3

- يحصل لما تعمل عملية على نوع مش مناسب

مثال:

```
const num = 5;  
num.toUpperCase();
```

هيديك: 

```
TypeError: num.toUpperCase is not a function
```

Range Error (خارج المدى) 4

- يحصل لما قيمة تكون خارج الحد المسموح لبعض الدوال

مثال:

```
const arr = new Array(-1);
```

هيديك: 

```
RangeError: Invalid array length
```

URI Error 5

- يحصل في دوال مثل:

```
decodeURI("%")
```

هيديك: 

```
URIError: URI malformed
```

نصائح مهمة

- `SyntaxError` → صحح الكود قبل التشغيل
- `ReferenceError` → تأكد من تعريف كل المتغيرات
- `TypeError` → تحقق من نوع البيانات قبل الاستخدام
- `RangeError` → راقب القيم وحدودها
- `URIError` → تأكد من الترميز قبل `decode/encode`

Task صغير

المطلوب:

- 1 اكتب كود يحتوي على **Reference Error**
- 2 اكتب كود يحتوي على **Type Error**
- 3 جرب التعامل معاهم باستخدام `try...catch`

When JS encounters something **unknown or wrong during execution**, it throws an **Error**.

- If you don't handle it, the code **stops running**.

1 Syntax Error

Happens when your code is **not understandable by JS**.

- Appears **before execution**

```
console.log("Hello"
```

✓ Output:

```
SyntaxError: missing ) after argument list
```

2 Reference Error

Happens when you try to **use a variable that is not defined**

```
console.log(name);
```

✓ Output:

```
ReferenceError: name is not defined
```

3 Type Error

Happens when you **perform an operation on the wrong data type**

```
const num = 5;
```

```
num.toUpperCase();
```

✓ Output:

```
TypeError: num.toUpperCase is not a function
```


4 Range Error

Happens when a value is **outside the allowed range** for some functions

```
const arr = new Array(-1);
```

✓ Output:

```
RangeError: Invalid array length
```

5 URI Error

Happens in functions like:

```
decodeURI("%");
```

✓ Output:

```
URIError: URI malformed
```

Tips

- **SyntaxError** → Fix the code before running
- **ReferenceError** → Make sure all variables are defined
- **TypeError** → Check the data type before using it
- **RangeError** → Watch the values and their limits
- **URIError** → Make sure encoding is correct before decode/encode

Small Task

Requirements:

- 1 Write code that produces a **Reference Error**

② Write code that produces a **Type Error**

③ Handle both using `try...catch`

Example:

```
// Reference Error
```

```
try {
```

```
    console.log(myVar);
```

```
} catch (error) {
```

```
    console.log("Caught Reference Error:", error.message);
```

```
}
```

```
// Type Error
```

```
try {
```

```
    const num = 10;
```

```
    num.toUpperCase();
```

```
} catch (error) {
```

```
    console.log("Caught Type Error:", error.message);
```

```
}
```

✓ Output:

Caught Reference Error: myVar is not defined

Caught Type Error: num.toUpperCase is not a function

Basic problem solving

Basic problem solving

أساسيات حل المشكلات

يعني إيه Problem Solving؟

حل المشكلات في البرمجة يعني:

- تأخذ مشكلة
 - تفهمها
 - تقسمها لخطوات صغيرة
 - تحوّل الخطوات دي لكود JavaScript
- البرمجة = تفكير منطقي + أدوات (JS)

خطوات حل أي مشكلة برمجية

1 فهم المشكلة

- المطلوب إيه؟
- المدخلات (Input) إيه؟
- الناتج (Output) إيه؟

مثال:

اعرض الأرقام الزوجية من 1 لـ 10

2 تقسيم المشكلة

- هنعهد من 1 لـ 10
- نختبر هل الرقم زوجي؟
- لو زوجي → نعرضه

3 اختيار الأدوات

المهمة	الأداة
تكرار	loop
شرط	if
تخزين	variable

4 كتابة الكود

```
for (let i = 1; i <= 10; i++) {  
  if (i % 2 === 0) {  
    console.log(i);  
  }  
}
```

```
}  
}
```

أمثلة شائعة في Problem Solving

مثال 1: حساب مجموع الأرقام

المطلوب: احسب مجموع الأرقام من 1 إلى 5

التفكير:

- متغير للمجموع
- لوب
- نجمع

```
let sum = 0;
```

```
for (let i = 1; i <= 5; i++) {  
    sum += i;  
}
```

```
console.log(sum); // 15
```


مثال 2: إيجاد أكبر رقم في Array

```
let numbers = [3, 7, 2, 9, 1];  
let max = numbers[0];  
  
for (let i = 1; i < numbers.length; i++) {  
    if (numbers[i] > max) {  
        max = numbers[i];  
    }  
}  
  
console.log(max); // 9
```

مثال 3: عدّ العناصر الزوجية

```
let arr = [1, 4, 6, 7, 9, 10];  
let count = 0;  
  
for (let i = 0; i < arr.length; i++) {  
    if (arr[i] % 2 === 0) {  
        count++;  
    }  
}  
  
console.log(count); // 3
```

مثال 4: Function تحل مشكلة

```
function isEven(num) {  
    return num % 2 === 0;  
}
```

```
console.log(isEven(4)); // true  
console.log(isEven(7)); // false
```

أنماط مشاكل لازم تتعود عليها

- Loop + Condition
- التعامل مع Arrays
- عدّ / تجميع
- مقارنة قيم
- Functions لحل مشكلة صغيرة

أخطاء شائعة في Problem Solving

- البدء بالكود قبل الفهم
- كتابة كود طويل بدل تقسيم المشكلة
- نسيان edge cases
- الاعتماد على copy-paste

اختر الإجابة الصحيحة (MCQ)

1 ما أول خطوة في حل أي مشكلة؟

- (A) كتابة الكود
- (B) اختيار loop
- (C) فهم المشكلة
- (D) استخدام function

2 أي أداة مناسبة للتكرار؟

- A) if
- B) loop
- C) function
- D) variable

3 ما ناتج الكود؟

```
let x = 0;
for (let i = 1; i <= 3; i++) {
  x += i;
}
console.log(x);
```

A) 3

B) 5

C) 6

D) 7

Problem solving means:

- Take a problem
- Understand it
- Break it into small steps
- Turn those steps into **JavaScript code**

Programming = Logical thinking + Tools (JS)

Steps to Solve Any Programming Problem

1 Understand the problem

- What is required?
- What is the input?
- What is the output?

Example:

Print all even numbers from 1 to 10

2 Break the problem

- Count from 1 to 10
- Check if the number is even
- If even → print it

3 Choose the right tools

Task	Tool
Repetition	loop
Condition	if
Storage	variable

4 Write the code

```
for (let i = 1; i <= 10; i++) {  
  if (i % 2 === 0) {  
    console.log(i);  
  }  
}
```

Common Examples in Problem Solving

Example 1: Sum of numbers

Calculate sum of numbers from 1 to 5

```
let sum = 0;  
  
for (let i = 1; i <= 5; i++) {  
  sum += i;  
}  
  
console.log(sum); // 15
```

Example 2: Find max number in Array

```
let numbers = [3, 7, 2, 9, 1];  
let max = numbers[0];
```

```
for (let i = 1; i < numbers.length; i++) {  
  if (numbers[i] > max) {  
    max = numbers[i];  
  }  
}  
  
console.log(max); // 9
```

Example 3: Count even numbers in Array

```
let arr = [1, 4, 6, 7, 9, 10];  
let count = 0;  
  
for (let i = 0; i < arr.length; i++) {  
  if (arr[i] % 2 === 0) {  
    count++;  
  }  
}  
  
console.log(count); // 3
```

Example 4: Function to solve a problem

```
function isEven(num) {
```



```
    return num % 2 === 0;
}

console.log(isEven(4)); // true
console.log(isEven(7)); // false
```

Patterns You Should Practice

- Loop + Condition
- Working with Arrays
- Counting / Summing
- Comparing values
- Functions for small problems

Common Mistakes in Problem Solving

- ✗ Start coding before understanding the problem
- ✗ Writing long code instead of breaking the problem
- ✗ Forgetting edge cases
- ✗ Copy-paste without understanding

MCQs

① What is the **first step** in solving any problem?

- A) Write code
- B) Choose a loop
- C) Understand the problem
- D) Use a function

② Which tool is suitable for repetition?

- A) if
- B) loop
- C) function
- D) variable

③ What is the output?

```
let x = 0;  
for (let i = 1; i <= 3; i++) {  
  x += i;  
}
```

```
}  
console.log(x);
```

A) 3

B) 5

C) 6

D) 7

Intro to DOM

Intro to DOM (Document Object Model)

خريطة رسم الصفحة

يعني إيه DOM؟

الـ DOM هو:

تمثيل الصفحة (HTML) في صورة شجرة كائنات (Objects)
عشان JavaScript تقدر:

- تقرأ
 - تعدّل
 - تتحكم في عناصر الصفحة
- يعني JS بتكلم الصفحة نفسها.

الفكرة ببساطة

:HTML

```
<h1>Hello</h1>  
<p>Welcome</p>
```

JS بتشوفهم كده:

```
document
├── html
│   └── body
│       ├── h1
│       └── p
```

ليه DOM مهم؟

من غير DOM:

- الصفحة ثابتة

مع DOM:

- تغيير النص
- تغيير الشكل
- التفاعل مع المستخدم
- بناء تطبيقات حقيقية

الوصول لعناصر الصفحة (Selecting Elements)

`getElementById` ¹

```
<h1 id="title">Hello</h1>
```

```
let title = document.getElementById("title");
```

querySelector (الأشهر) 2

```
document.querySelector("h1");  
document.querySelector(".box");  
document.querySelector("#title");
```

يستخدم نفس selectors بتاعة CSS

querySelectorAll 3

```
let items = document.querySelectorAll("li");
```

يرجع **NodeList** (شبه Array)

تعديل المحتوى

تغيير النص

```
title.textContent = "Hello DOM";
```

تغيير HTML

```
title.innerHTML = "<span>Hi</span>";
```

تعديل الـ Styles

```
title.style.color = "red";  
title.style.fontSize = "32px";
```

inline style

إضافة / حذف Classes

```
title.classList.add("active");  
title.classList.remove("active");  
title.classList.toggle("active");
```

Events (ببساطة)

DOM يسمح للـ JS تسمع للأحداث:

```
let btn = document.querySelector("button");  
  
btn.addEventListener("click", function () {  
    alert("Button clicked!");  
});
```


مثال كامل بسيط

```
<button id="btn">Click me</button>  
<p id="text">Hello</p>
```

```
let btn = document.getElementById("btn");  
let text = document.getElementById("text");
```

```
btn.addEventListener("click", function () {  
    text.textContent = "You clicked the button!";  
});
```

أخطاء شائعة في DOM

- استخدام JS قبل تحميل الصفحة
- نسيان # أو . في querySelector
- استخدام innerHTML بدون داعي
- التعديل على style بدل class

اختر الإجابة الصحيحة (MCQ)

1 DOM اختصار لـ؟

- A) Data Object Model
- B) Document Object Model

C) Dynamic Object Method

D) Document Order Model

2) أي كود يختار عنصر بـ id؟

A) `document.querySelector(".title")`

B) `document.getElement("title")`

C) `document.getElementById("title")`

D) `document.querySelectorAll("#title")`

3) أي Selector بيقبل CSS syntax؟

A) `getElementById`

B) `querySelector`

C) `innerHTML`

D) `classList`

The **DOM** (Document Object Model) is:

- A representation of the HTML page as a **tree of objects**
- Allows **JavaScript** to:
 - Read
 - Modify
 - Control page elements

Basically, JS talks to the page itself.

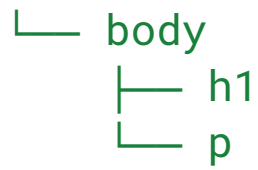
Simple Idea

HTML:

```
<h1>Hello</h1>  
<p>Welcome</p>
```

JS sees it as:

```
document  
└─ html
```



Why DOM is Important?

- Without DOM → page is static
- With DOM → you can:
 - Change text
 - Change styles
 - Handle user interaction
 - Build real applications

Selecting Elements

1 getElementById

```
<h1 id="title">Hello</h1>
```

```
let title = document.getElementById("title");
```

2 querySelector (most used)

```
document.querySelector("h1");  
document.querySelector(".box");  
document.querySelector("#title");
```

Works like CSS selectors

3 querySelectorAll

```
let items = document.querySelectorAll("li");
```

Returns a **NodeList** (similar to Array)

Modify Content

Change text

```
title.textContent = "Hello DOM";
```

Change HTML

```
title.innerHTML = "<span>Hi</span>";
```

Change styles

```
title.style.color = "red";  
title.style.fontSize = "32px";
```

Inline style

Add / Remove / Toggle classes

```
title.classList.add("active");  
title.classList.remove("active");  
title.classList.toggle("active");
```

Events (Simple)

DOM allows JS to **listen to events**:

```
let btn = document.querySelector("button");
```

```
btn.addEventListener("click", function () {  
    alert("Button clicked!");  
});
```

Complete simple example:

```
<button id="btn">Click me</button>  
<p id="text">Hello</p>
```

```
let btn = document.getElementById("btn");  
let text = document.getElementById("text");  
  
btn.addEventListener("click", function () {  
    text.textContent = "You clicked the button!";  
});
```

Common DOM Mistakes

- ✗ Using JS before the page loads
- ✗ Forgetting `#` or `.` in `querySelector`
- ✗ Using `innerHTML` unnecessarily
- ✗ Changing `style` instead of using `class`

MCQs

1 DOM stands for?

- A) Data Object Model
- B) Document Object Model
- C) Dynamic Object Method
- D) Document Order Model

2 Which code selects an element by id?

- A) `document.querySelector(".title")`
- B) `document.getElementById("title")`
- C) `document.getElementById("title")`
- D) `document.querySelectorAll("#title")`

3 Which selector accepts **CSS syntax**?

- A) `getElementById`
- B) `querySelector`
- C) `innerHTML`
- D) `classList`

DOM Selection & Manipulation

DOM Selection & Manipulation

يعني إيه Selection & Manipulation؟

● **Selection** → اختيار عناصر من الصفحة

● **Manipulation** → تعديل العناصر دي (نص - شكل - Classes

(Attributes

JavaScript من غير DOM = مالوش تأثير على الصفحة

أولاً: DOM Selection (اختيار العناصر)

getElementById 

```
<h1 id="title">Hello</h1>
```

```
const title = document.getElementById("title");
```

✓ سريع

✗ بيختار عنصر واحد بس

getElementsByTagName²

```
const items =  
document.getElementsByClassName("item");
```

يرجع **HTMLCollection**

✗ مش Array حقيقي

getElementsByTagName³

```
const paragraphs =  
document.getElementsByTagName("p");
```

querySelector⁴ (الأهم)

```
document.querySelector("h1");  
document.querySelector(".item");  
document.querySelector("#title");
```

✓ أسهل

✓ يدعم CSS selectors

✗ يرجع أول عنصر بس

querySelectorAll 5

```
const list = document.querySelectorAll(".item");
```

يرجع **NodeList**

✓ ينفع نعمل عليه loop

ثانيًا: DOM Manipulation (التعديل)

1 تعديل النص

```
element.textContent = "New text";
```

✓ آمن

✗ لا يقبل HTML

2 تعديل HTML

```
element.innerHTML = "<strong>Hello</strong>";
```

خطر لو الداتا جاية من المستخدم

3 تعديل الـ Style

```
element.style.color = "blue";  
element.style.backgroundColor = "lightgray";
```

Inline style

4 Classes (أفضل ممارسة)

```
element.classList.add("active");  
element.classList.remove("active");  
element.classList.toggle("active");
```

✓ أنظف

✓ أسهل في التحكم

5 Attributes

```
img.setAttribute("src", "image.png");  
img.getAttribute("alt");  
img.removeAttribute("title");
```

أشهر Attributes:

- src
- href
- value
- disabled

6 إنشاء وحذف عناصر

إنشاء عنصر

```
const div = document.createElement("div");  
div.textContent = "New element";  
document.body.appendChild(div);
```

حذف عنصر

```
element.remove();
```

Loop على عناصر

```
items.forEach(item => {  
  item.classList.add("box");  
});
```

ينفع مع `querySelectorAll`

مثال تطبيقي صغير

```
<button id="btn">Change</button>  
<p class="text">Hello</p>
```

```
const btn = document.getElementById("btn");  
const text = document.querySelector(".text");
```

```
btn.addEventListener("click", () => {  
  text.textContent = "Text changed!";  
  text.classList.toggle("active");  
});
```

أخطاء شائعة

- استخدام `innerHTML` بدل `textContent`
- نسيان إن `getElements` يرجع `Collection`
- التعديل على `style` بدل `classes`

- اختيار عنصر مش موجود → error

اختر الإجابة الصحيحة (MCQ)

1 أي Selector يدعم CSS؟

- A) getElementById
- B) getElementsByTagName
- C) querySelector
- D) createElement

2 الفرق بين querySelector و querySelectorAll؟

- A) مفيش فرق
- B) Array بترجع All
- C) يرجع عنصر واحد Selector
- D) بترجع أول عنصر All

3 أفضل طريقة لتغيير الشكل؟

- A) `element.style`
- B) `element.innerHTML`
- C) `element.classList`
- D) `element.textContent`

What is Selection & Manipulation?

- **Selection** → Choosing elements from the page
- **Manipulation** → Changing those elements (text, style, classes, attributes)

JavaScript without DOM = **no effect on the page**

1 DOM Selection (Choosing Elements)

1 getElementById

```
<h1 id="title">Hello</h1>
```

```
const title = document.getElementById("title");
```

✓ Fast

✗ Selects **only one element**

2) `getElementsByClassName`

```
const items =  
document.getElementsByClassName("item");
```

- Returns **HTMLCollection**

✗ Not a real Array

3) `getElementsByTagName`

```
const paragraphs =  
document.getElementsByTagName("p");
```

4) `querySelector` (most used)

```
document.querySelector("h1");  
document.querySelector(".item");  
document.querySelector("#title");
```

✓ Easy

✓ Supports CSS selectors

✗ Returns **only the first element**

5 `querySelectorAll`

```
const list = document.querySelectorAll(".item");
```

- Returns **NodeList**

- ✓ Can loop over it

2 DOM Manipulation (Changing Elements)

1 Change text

```
element.textContent = "New text";
```

✓ Safe

✗ Doesn't accept HTML

2 Change HTML

```
element.innerHTML = "<strong>Hello</strong>";
```

Dangerous if data comes from user

3 Change Style

```
element.style.color = "blue";  
element.style.backgroundColor = "lightgray";
```

Inline style

4 Classes (Best Practice)

```
element.classList.add("active");
```

```
element.classList.remove("active");  
element.classList.toggle("active");
```

✓ Cleaner

✓ Easier to manage

5 Attributes

```
img.setAttribute("src", "image.png");  
img.getAttribute("alt");  
img.removeAttribute("title");
```

Common Attributes:

`src, href, value, disabled`

6 Create & Remove Elements

Create element

```
const div = document.createElement("div");
```

```
div.textContent = "New element";  
document.body.appendChild(div);
```

Remove element

```
element.remove();
```

Loop over elements

```
items.forEach(item => {  
    item.classList.add("box");  
});
```

Works with `querySelectorAll`

Practical Example

```
<button id="btn">Change</button>  
<p class="text">Hello</p>
```

```
const btn = document.getElementById("btn");
```

```
const text = document.querySelector(".text");

btn.addEventListener("click", () => {
  text.textContent = "Text changed!";
  text.classList.toggle("active");
});
```

Common Mistakes

- ✗ Using `innerHTML` instead of `textContent`
- ✗ Forgetting that `getElements` returns a Collection
- ✗ Changing `style` instead of `class`
- ✗ Selecting a non-existing element → Error

MCQs

① Which selector supports CSS syntax?

- A) `getElementById`
- B) `getElementsByName`

C) `querySelector`

D) `createElement`

2) Difference between `querySelector` & `querySelectorAll`?

A) No difference

B) `All` returns an Array

C) `Selector` returns **one element**

D) `All` returns first element

3) Best way to change the appearance?

A) `element.style`

B) `element.innerHTML`

C) `element.classList`

D) `element.textContent`

Events

Events

(JavaScript الأحداث في)

يعني إيه Event؟

الـ **Event** هو أي حاجة المستخدم يعملها في الصفحة، زي:

- كليك
- كتابة
- تحريك الماوس
- تحميل الصفحة

JavaScript بتسمع للأحداث دي وتنقذ كود معين وقت ما تحصل

الفكرة ببساطة

المستخدم يعمل حاجة → يحصل JavaScript → Event يرد عليها

أشهر أنواع الـ Events

Event	بيحصل إمتى
click	عند الضغط
submit	عند إرسال الفورم
input	عند الكتابة
change	لما القيمة تتغير
mouseover	الماوس يعدي
keydown	ضغط زر
load	الصفحة تفتح
scroll	اسكرول

طرق التعامل مع Events

الطريقة القديمة (غير مفضلة)

```
<button onclick="sayHello()">Click</button>
```

مشاكل:

- كود HTML + JS مع بعض
- صعب الصيانة

الطريقة الصحيحة: `addEventListener` ✓

```
button.addEventListener("click", () => {  
  console.log("Clicked!");  
});
```

✓ أنظف

✓ ينفذ أكثر من Event على نفس العنصر

مثال بسيط

```
<button id="btn">Click me</button>
```

```
const btn = document.getElementById("btn");
```

```
btn.addEventListener("click", () => {  
  alert("Button clicked");  
});
```

Event Object (e)

كل Event يبعث object فيه معلومات عنه

```
button.addEventListener("click", (e) => {  
  console.log(e);  
});
```

أشهر استخدامات:

```
e.target  
e.preventDefault()
```

منع السلوك الافتراضي

مثال: منع الفورم من الريلود

```
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
});
```

Input مع Events

```
input.addEventListener("input", () => {  
  console.log(input.value);  
});
```

✔ يشتغل مع كل كتابة

Keyboard Events

```
document.addEventListener("keydown", (e) => {  
  console.log(e.key);  
});
```

Mouse Events

```
box.addEventListener("mouseover", () => {  
  box.classList.add("hover");  
});
```

Event Loop داخل Loop

```
buttons.forEach(btn => {  
  btn.addEventListener("click", () => {  
    console.log(btn.textContent);  
  });  
});
```

أخطاء شائعة

- نسيان `addEventListener`
- كتابة اسم Event غلط (`onclick` بدل `click`)
- نسيان `preventDefault`
- استخدام inline events

- استخدام event قبل اختيار العنصر

اختر الإجابة الصحيحة (MCQ)

1 أفضل طريقة لإضافة Event؟

- A) onclick
- B) onClick
- C) addEventListener
- D) click()

2 أي Event مناسب للفورم؟

- A) click
- B) submit
- C) load
- D) keydown

3) لمنع Reload الفورم نستخدم؟

- A) stop
- B) prevent
- C) preventDefault
- D) return false

An **Event** is anything a user does on the page, like:

- Click
- Typing
- Moving the mouse
- Page loading

JavaScript listens to these events and runs code when they happen

Simple idea:

User does something → Event happens → JavaScript reacts

Common Event Types

Event	Happens When
click	When clicking

submit	When submitting a form
input	While typing
change	When value changes
mouseover	Mouse moves over an element
keydown	Pressing a key
load	Page loads
scroll	Page scrolls

Ways to Handle Events

Old way (not recommended)

```
<button onclick="sayHello()">Click</button>
```

Problems:

- HTML + JS mixed
- Hard to maintain

✅ **Correct way: `addEventListener`**

```
button.addEventListener("click", () => {  
    console.log("Clicked!");  
});
```

✓ Cleaner

✓ Can attach **multiple events** on the same element

Simple Example

```
<button id="btn">Click me</button>
```

```
const btn = document.getElementById("btn");
```

```
btn.addEventListener("click", () => {  
    alert("Button clicked");  
});
```

Event Object (e)

Every event sends an **object** with info about it:

```
button.addEventListener("click", (e) => {  
    console.log(e);  
});
```

Common uses:

- `e.target` → element that triggered the event
- `e.preventDefault()` → prevent default behavior

Prevent default example:

```
form.addEventListener("submit", (e) => {  
    e.preventDefault(); // stop reload  
});
```

Events with Input

```
input.addEventListener("input", () => {  
    console.log(input.value);  
});
```

✓ Works on every keystroke

Keyboard Events

```
document.addEventListener("keydown", (e) => {  
    console.log(e.key);  
});
```

Mouse Events

```
box.addEventListener("mouseover", () => {
```

```
    box.classList.add("hover");  
  });
```

Event Loop inside Loop

```
buttons.forEach(btn => {  
    btn.addEventListener("click", () => {  
        console.log(btn.textContent);  
    });  
});
```

Common Mistakes

- ❌ Forgetting `addEventListener`
- ❌ Wrong event name (`onclick` instead of `click`)
- ❌ Forgetting `preventDefault`
- ❌ Using inline events
- ❌ Using the element before selecting it

MCQs

1) Best way to add an Event?

- A) onclick
- B) onClick
- C) addEventListener
- D) click()

2) Which event is suitable for forms?

- A) click
- B) submit
- C) load
- D) keydown

3) To prevent form reload, we use?

- A) stop
- B) prevent
- C) preventDefault
- D) return false

LocalStorage Basics

LocalStorage Basics

التخزين في المتصفح

يعني إيه LocalStorage؟

الـ **LocalStorage** طريقة نخرّن بيها بيانات في المتصفح نفسه
البيانات:

- بتفضل محفوظة حتى بعد قفل المتصفح
- مربوطة بالدومين (الموقع)

الفكرة ببساطة

نخزن بيانات بسيطة (key → value)
ونرجعها بعدين في أي وقت

زي:

- اسم المستخدم
- ثيم الموقع (دارك / لايت)
- مهام ToDo
- إعدادات بسيطة

LocalStorage = Key / Value

key → value

القيمة دائمًا String

حفظ بيانات

```
localStorage.setItem("username", "Youssef");
```

قراءة بيانات

```
const name = localStorage.getItem("username");  
console.log(name);
```

حذف عنصر

```
localStorage.removeItem("username");
```

مسح كل البيانات

```
localStorage.clear();
```

مثال كامل (حفظ اسم)

```
const input = document.querySelector("input");
const button = document.querySelector("button");

button.addEventListener("click", () => {
  localStorage.setItem("name", input.value);
});
```

قراءة البيانات عند تحميل الصفحة

```
const savedName = localStorage.getItem("name");

if (savedName) {
  input.value = savedName;
}
```

تخزين Arrays & Objects

للازم نحولهم لـ JSON

حفظ

```
const user = {
  name: "Youssef",
  age: 25
};
```

```
localStorage.setItem("user",  
JSON.stringify(user));
```

قراءة

```
const data =  
JSON.parse(localStorage.getItem("user"));  
console.log(data.name);
```

أخطاء شائعة

- نسيان `JSON.stringify`
- محاولة تخزين function
- الاعتماد على LocalStorage كـ Database
- تخزين بيانات حساسة (passwords) ❌
- نسيان التحقق من وجود البيانات

LocalStorage vs SessionStorage

SessionStorage	LocalStorage	Feature
❌	✅	ييفضل بعد قفل المتصفح

5MB~	5MB~	سعة التخزين
ضعيف	ضعيف	الأمان
جلسة مؤقتة	إعدادات	الاستخدام

اختر الإجابة الصحيحة (MCQ)

1 LocalStorage ييخزن البيانات؟

- (A) على السيرفر
- (B) في قاعدة بيانات
- (C) في المتصفح
- (D) في الكاش

2 نوع البيانات المخزنة؟

- A) أي نوع
- B) فقط Object
- C) فقط String
- D) فقط Number

3 تخزين Object صح؟

A) `localStorage.setItem("u", user);`

B) `localStorage.setItem("u",
JSON.stringify(user));`

C) `localStorage.u = user;`

D) `localStorage.push(user);`

LocalStorage is a way to store data **in the browser itself**.

- Data **remains saved even after closing the browser**
- Data is **linked to the domain (website)**

Simple idea:

Store simple data (key → value) and retrieve it anytime

Examples:

- Username
- Site theme (dark/light)
- ToDo tasks
- Simple settings

LocalStorage = Key / Value

Value is **always a String**

Saving Data


```
localStorage.setItem("username", "Youssef");
```

Reading Data

```
const name = localStorage.getItem("username");  
console.log(name);
```

Removing an Item

```
localStorage.removeItem("username");
```

Clear All Data

```
localStorage.clear();
```

Complete Example (Save Name)

```
const input = document.querySelector("input");  
const button = document.querySelector("button");  
  
button.addEventListener("click", () => {  
    localStorage.setItem("name", input.value);  
});
```

Read data on page load:

```
const savedName = localStorage.getItem("name");

if (savedName) {
  input.value = savedName;
}
```

Storing Arrays & Objects

Must convert to **JSON**

Save:

```
const user = { name: "Youssef", age: 25 };
localStorage.setItem("user",
JSON.stringify(user));
```

Read:

```
const data =
JSON.parse(localStorage.getItem("user"));
console.log(data.name);
```

Common Mistakes

- ✗ Forgetting `JSON.stringify`
- ✗ Trying to store a function
- ✗ Using LocalStorage as a database
- ✗ Storing sensitive data (passwords ✗)
- ✗ Forgetting to check if data exists

LocalStorage vs SessionStorage

Feature	LocalStorage	SessionStorage
Persists after closing browser	✓	✗
Storage size	~5MB	~5MB
Security	Weak	Weak
Usage	Settings	Temporary session

MCQs

1) Where does LocalStorage store data?

- A) On the server
- B) In a database

C) In the browser

D) In the cache

2) What type of data is stored?

A) Any type

B) Object only

C) String only

D) Number only

3) Correct way to store an object?

A) `localStorage.setItem("u", user);`

B) `localStorage.setItem("u",
JSON.stringify(user));`

C) `localStorage.u = user;`

D) `localStorage.push(user);`

How JS Interacts with HTML & CSS

How JS Interacts with HTML & CSS

1 العلاقة بين HTML, CSS و JavaScript

- **HTML:** هو الهيكل الأساسي للصفحة، زي العمود الفقري. يحدد العناصر اللي موجودة (مثل أزرار، صور، نصوص...).
- **CSS:** هو اللي يحدد شكل الصفحة، الألوان، الخطوط، المسافات... زي الملابس اللي البنية بتلبسها.
- **JavaScript:** هو اللي بيخلي الصفحة حية، يعني يغير المحتوى، يضيف أحداث، يتحكم في العناصر ويخلي المستخدم يتفاعل مع الصفحة.

مثال توضيحي:

```
<DOCTYPE html!>
```

```
<html>
```

```
<head>
```

```
<title>مثال تفاعل</title>
```

```
<style>
```

```
        } myButton#
;background-color: lightblue
        ;padding: 10px 20px
        ;border: none
        ;cursor: pointer
    {
</style>
</head>
<body>
    <h1/>!مرحبًا بك!<"h1 id="title>
<button/>اضغط علي<"button id="myButton>

    <script>

        const button =
;('document.getElementById('myButton

        const title =
;('document.getElementById('title
```

```

    } <= ( ) , 'button.addEventListener('click
; "لقد ضغطت على الزر!" = title.textContent

    ; "title.style.color = "red

    ; ({
</script>

</body>

</html>

```

شرح المثال:

- `document.getElementById('myButton')`: جلبنا عنصر الزر من الصفحة.
- `addEventListener('click', ...)`: JS يستمع لحدث النقر على الزر.
- `title.textContent = "..."`: غيرنا نص عنصر الـ HTML.
- `title.style.color = "red"`: غيرنا شكل العنصر (CSS) مباشرة من JS.

2 تغيير المحتوى الديناميكي (HTML)

JavaScript ممكن يغير أي حاجة في الصفحة وقت التشغيل.

مثال:

```
;( 'const p = document.querySelector('p
    = p.innerHTML "هذا النص تم تغييره بواسطة
    ;"!JavaScript
```

- **innerHTML** يغير كل المحتوى داخل العنصر.
- **textContent** يغير النص فقط بدون أكواد HTML داخل العنصر.

3 تغيير الأنماط (CSS) ديناميكيًا

ممكن تغيير خصائص CSS مباشرة أو تضيف/remove كلاس:

مثال تغيير ستايل مباشر:

```
;( 'const box = document.querySelector('.box
    ;'box.style.backgroundColor = 'yellow
    ;'box.style.width = '200px
```

مثال بإضافة/إزالة كلاس:

```
/* CSS */

} highlight.

;border: 2px solid red

;background-color: lightyellow

{

;('const box = document.querySelector('.box
تضيف الكلاس // ;('box.classList.add('highlight
تحدفه // ;('box.classList.remove('highlight
تضيف أو // ;('box.classList.toggle('highlight
تحدف حسب وجوده
```

نصيحة: استخدام `classList` أفضل من تغيير كل ستايل مباشرة لأنه يحافظ على التنسيق الأصلي للصفحة.

4) التفاعل مع المستخدم

JavaScript يجعل الصفحة تتفاعل مع أحداث المستخدم مثل النقر، التحريك، الكتابة...

مثال: تغيير النص أثناء الكتابة

```
input type="text" id="nameInput">
    <"اكتب اسمك هنا"=placeholder

    <p id="greet"></p>

    <script>

        const input =
        ;('document.getElementById('nameInput

        const greet =
        ;('document.getElementById('greet

    } <= ( ) , 'input.addEventListener('input
    ;`!{input.value}$ مرحبًا` = greet.textContent

    ;({

    <script/>
```

- كل حرف المستخدم يكتبه، يظهر مباشرة في الفقرة.
- هذا مثال لتفاعل **Realtime** بين JS وHTML.

5 نصائح مهمة

1. حاول فصل **HTML, CSS, JS** بقدر الإمكان، بس JS تتفاعل مع HTML وCSS عن طريق **DOM** و **class**.
2. استخدم **querySelector** و **querySelectorAll** بدل **getElementById** أو **getElementsByClassName** لأنها أكثر مرونة.
3. لتغيير شكل العناصر، استخدم **classList** بدل تعديل **style** مباشرة.
4. دائمًا اختبر تغييراتك في Console قبل تطبيقها على الصفحة.
5. استغل (click, input, mouseover, scroll) **Events** (...) للتحكم في التفاعل بدلًا من تعديل كل حاجة مباشرة.

1 Relationship Between HTML, CSS, and JavaScript

- **HTML:** The backbone of the page, defines elements like buttons, images, texts...
- **CSS:** Controls the style, colors, fonts, spacing... like the “clothes” the page wears.
- **JavaScript:** Makes the page interactive—changes content, handles events, manipulates elements.

Example:

```
<!DOCTYPE html>

<html>

<head>

  <title>JS Interaction Example</title>

  <style>

    #myButton {

      background-color: lightblue;

      padding: 10px 20px;
```

```
        border: none;

        cursor: pointer;
    }
</style>
</head>
<body>

    <h1 id="title">Welcome!</h1>

    <button id="myButton">Click Me</button>


    <script>

        const button =
document.getElementById('myButton');

        const title =
document.getElementById('title');


        button.addEventListener('click', () => {

            title.textContent = "You clicked the
button!";

            title.style.color = "red";
```

```
        });  
    </script>  
</body>  
</html>
```

Explanation:

- `document.getElementById( 'myButton' )`: grabs the button element from HTML.
- `addEventListener( 'click', ... )`: JS listens for a click event.
- `title.textContent = "..."`: changes the HTML text.
- `title.style.color = "red"`: changes the element's CSS directly.

2 Changing HTML Content Dynamically

JS can change almost anything in the page at runtime.

Example:

```
const p = document.querySelector('p');  
  
p.innerHTML = "This text was changed using  
JavaScript!";
```

- `innerHTML` changes all content inside the element (including HTML).
- `textContent` changes only the text, ignoring HTML tags.

3 Changing CSS Styles Dynamically

You can either change CSS properties directly or add/remove classes:

Direct style example:

```
const box = document.querySelector('.box');  
  
box.style.backgroundColor = 'yellow';  
  
box.style.width = '200px';
```

Using classes example:


```
/* CSS */

.highlight {
  border: 2px solid red;
  background-color: lightyellow;
}

const box = document.querySelector('.box');
box.classList.add('highlight');    // add class
box.classList.remove('highlight'); // remove
class
box.classList.toggle('highlight'); // add/remove
depending on existence
```

Tip: Using `classList` is better than changing styles directly—it keeps your original styles intact.

4 Interacting with Users

JS allows the page to react to user events like click, hover, typing, scroll...

Example: Updating text as the user types

```
<input type="text" id="nameInput"
placeholder="Type your name">
```

```
<p id="greet"></p>
```

```
<script>
```

```
  const input =
document.getElementById('nameInput');
```

```
  const greet =
document.getElementById('greet');
```

```
  input.addEventListener('input', () => {
    greet.textContent = `Hello,
${input.value}!`;
  });
```

```
</script>
```

- Every character typed instantly updates the paragraph.
- This is an example of **real-time interaction** between JS and HTML.

5 Important Tips

1. Keep **HTML, CSS, and JS separate** as much as possible, but JS interacts with HTML/CSS via **DOM** and **classes**.
2. Prefer `querySelector` and `querySelectorAll` over `getElementById` or `getElementsByClassName`—more flexible.
3. Use `classList` to change styles instead of modifying `style` directly.
4. Test your changes in the console before applying them to the page.
5. Use **events** (click, input, mouseover, scroll...) to control interactions rather than manipulating everything manually.

Exercise: Interactive To-Do List

Exercise: Interactive To-Do List

هدف التمرين

بناء تطبيق بسيط لإدارة المهام يخلّي الطالب:

- يفهم التفاعل الحقيقي مع المستخدم
- يربط JavaScript بالـ DOM
- يستخدم LocalStorage بشكل عملي
- يفكر بمنطق مش حفظ

فكرة المشروع

المستخدم يكتب مهمة → يضيفها → تظهر في القائمة
يقدر:

- يعلّمها كمكتملة
- يحذفها
- يلاقيها محفوظة حتى بعد Refresh

المطلوبة Features

1. إضافة مهمة جديدة
2. عرض المهام في قائمة
3. حذف مهمة
4. تحديد مهمة كمكتملة (Done)
5. حفظ المهام في LocalStorage
6. تحميل المهام عند فتح الصفحة

Structure (HTML)

عناصر أساسية:

- Input لكتابة المهمة
- Button للإضافة
- List لعرض المهام

```
<input type="text" placeholder="New task" />  
<button>Add</button>  
<ul></ul>
```

Step 1: قراءة الإدخال

```
const input = document.querySelector("input");  
const button = document.querySelector("button");  
const list = document.querySelector("ul");
```

Step 2: إضافة Event

```
button.addEventListener("click", addTask);
```

Step 3: إضافة مهمة لـ DOM

```
function addTask() {  
  const text = input.value;  
  
  if (text === "") return;  
  
  const li = document.createElement("li");  
  li.textContent = text;  
  
  list.appendChild(li);  
  input.value = "";  
}
```

Step 4: حذف مهمة

```
li.addEventListener("click", () => {  
  li.remove();  
});
```

(تقدر تخلي الحذف بزرار مستقل)

Step 5: تعليم المهمة كمكتملة

```
li.addEventListener("dblclick", () => {  
  li.classList.toggle("done");  
});
```

```
.done {  
  text-decoration: line-through;  
  opacity: 0.6;  
}
```

Step 6: LocalStorage Logic

تخزين المهام

```
function saveTasks() {  
  const tasks = [];  
  
  document.querySelectorAll("li").forEach(li =>  
  {
```



```
    tasks.push({
      text: li.textContent,
      done: li.classList.contains("done")
    });
  });
```

```
  localStorage.setItem("tasks",
JSON.stringify(tasks));
}
```

تحميل المهام

```
function loadTasks() {
  const tasks =
JSON.parse(localStorage.getItem("tasks")) || [];

  tasks.forEach(task => {
    const li = document.createElement("li");
    li.textContent = task.text;

    if (task.done) li.classList.add("done");

    list.appendChild(li);
  });
}
```

اربط التخزين بالأحداث

```
button.addEventListener("click", () => {  
    addTask();  
    saveTasks();  
});
```

أخطاء شائعة

- نسيان حفظ البيانات بعد التعديل
- عدم التحقق من input فاضي
- تخزين HTML بدل البيانات
- استخدام innerHTML بشكل خاطئ

Build a **simple task management app** that helps the student:

- Understand real user interaction
- Connect **JavaScript with the DOM**
- Use **LocalStorage** practically
- Think logically, not just store data

Project Idea

User types a task → adds it → it appears in the list.

The user can:

- Mark it as **done**
- Delete it
- Keep it **saved even after refresh**

Required Features

1. Add a new task
2. Display tasks in a list
3. Delete a task
4. Mark a task as done
5. Save tasks in **LocalStorage**
6. Load tasks on page load

HTML Structure

Essential elements:

- Input to type task

- Button to add
- List to display tasks

```
<input type="text" placeholder="New task" />
<button>Add</button>
<ul></ul>
```

Step 1: Read Input

```
const input = document.querySelector("input");
const button = document.querySelector("button");
const list = document.querySelector("ul");
```

Step 2: Add Event

```
button.addEventListener("click", addTask);
```

Step 3: Add Task to DOM

```
function addTask() {
  const text = input.value;
```

```
if (text === "") return;

const li = document.createElement("li");
li.textContent = text;

list.appendChild(li);
input.value = "";
}
```

Step 4: Delete Task

```
li.addEventListener("click", () => {
  li.remove();
});
```

(You can also add a separate delete button instead of clicking the task itself)

Step 5: Mark Task as Done

```
li.addEventListener("dblclick", () => {
  li.classList.toggle("done");
});
```

```
.done {
```

```
    text-decoration: line-through;
    opacity: 0.6;
}
```

Step 6: LocalStorage Logic

Save Tasks:

```
function saveTasks() {
    const tasks = [];

    document.querySelectorAll("li").forEach(li =>
    {
        tasks.push({
            text: li.textContent,
            done: li.classList.contains("done")
        });
    });

    localStorage.setItem("tasks",
    JSON.stringify(tasks));
}
```

Load Tasks:

```
function loadTasks() {
```

```
const tasks =
JSON.parse(localStorage.getItem("tasks")) || [];

tasks.forEach(task => {
  const li = document.createElement("li");
  li.textContent = task.text;

  if (task.done) li.classList.add("done");

  list.appendChild(li);
});
}
```

Link Storage with Events

```
button.addEventListener("click", () => {
  addTask();
  saveTasks();
});
```

Common Mistakes

- ✗ Forgetting to save after editing tasks
- ✗ Not checking for empty input

✗ Storing HTML instead of data

✗ Misusing `innerHTML`

Task

Task: Personal Expense Tracker

Goal

بناء تطبيق لإدارة النفقات الشخصية بحيث:

- يسجل المستخدم المصروفات والإيرادات
- يعرض قائمة بالمعاملات
- يحسب الإجمالي (رصيد)
- يحفظ البيانات حتى بعد غلق الصفحة (LocalStorage)

Features

1. Add Transaction

- إدخال:
 - Description (الوصف)
 - Amount (المبلغ) → Expense = سالب, Income = موجب
- زر لإضافة المعاملة للقائمة

2. Display Transactions

- عرض كل المعاملات في قائمة
- كل عنصر يعرض: Description + Amount + Delete button
- تصنيف اللون: أحمر للمصروفات، أخضر للإيرادات

3. Delete Transaction

- إزالة أي معاملة من القائمة
- تحديث الرصيد الإجمالي مباشرة

4. Balance Calculation

- حساب:
 - Total Balance
 - Total Income
 - Total Expenses

5. Persistent Storage

- حفظ جميع البيانات في LocalStorage
- تحميل البيانات عند فتح الصفحة

6. Responsive Design

○ التطبيق يعمل بشكل جيد على الموبايل والكمبيوتر

Optional Enhancements

- Filter: Income / Expense / All
- Monthly / Weekly summary
- (Charts (Bar / Pie
- Animation عند إضافة أو حذف المعاملة

Suggested Structure

- **HTML:** input fields, add button, transactions list, summary section
- **CSS:** styling, responsive layout, color coding
- **JS:** DOM manipulation, Event handling, LocalStorage logic

Build a **Personal Expense Tracker** app so that:

- The user can record **expenses and income**
- Display a **list of transactions**
- Calculate the **total balance**
- Keep data **persistent using LocalStorage**, even after closing the page

Features

1 Add Transaction

Inputs:

- **Description** → text describing the transaction
- **Amount** → number (negative = Expense, positive = Income)

- **Add button** → adds the transaction to the list

2 Display Transactions

- Show all transactions in a list
- Each transaction shows: **Description + Amount + Delete button**
- Color coding: **Red = Expense, Green = Income**

3 Delete Transaction

- Remove any transaction from the list
- Update the total balance immediately

4 Balance Calculation

Calculate:

- **Total Balance**
- **Total Income**
- **Total Expenses**

5 Persistent Storage

- Save all transactions in **LocalStorage**
- Load data on page load

6 Responsive Design

- App should work well on **mobile and desktop**

Optional Enhancements

- **Filter:** Income / Expense / All
- **Summary:** Monthly / Weekly totals
- **Charts:** Bar or Pie charts
- **Animation** when adding or deleting a transaction

Suggested Structure

HTML:

- Input fields (description, amount)
- Add button
- Transaction list

- Summary section (balance, income, expenses)

CSS:

- Styling & responsive layout
- Color coding (red/green)

JS:

- DOM manipulation
- Event handling (add/delete transaction)
- LocalStorage logic